9.定时器与PWM

一、概念理解

在讲定时器之前先介绍前序内容——时钟树

1、时钟树

- 什么是时钟？什么是时钟树？

时钟是单片机的脉搏，是系统工作的同步节拍。每跳动一下，整个单片机的各个电路就同步的动作一下时钟信号是周期性的脉冲信号。在芯片内部是各种的逻辑门电路，如果不统一时序，会导致不同门电路的运算之间出现延迟，进而导致后续的寄存器出差，程序出错。所以有一个同步的信号就显得十分重要。 **同时**STM32有很多外设器件，每个器件的时钟信号不一样，所以要将一个固定的信号频率进行倍频/分频处理，达到每个外设需要的频率。**时钟信号的分频就像树的分支一样，这就是时钟树。**

- 什么是时钟频率？

指的是处理器或其他数字电路每秒钟振荡的次数，单位是赫兹（Hz），通常以 MHz（兆赫， Hz）来表示。例如，时钟频率为 1 GHz 的CPU，每秒钟振荡 10亿次。时钟频率直接决定了处理器执行指令的基本速度，但并不是唯一影响性能的因素。

- STM32F1 时钟介绍

HSI（内部高速时钟）: 时钟信号由内部RC震荡电路提供，时钟频率为8 MHz，但是这个时钟频率会随着温度产生漂移，很不稳定，所以一般不使用此时钟信号。

LSI（内部低速时钟）: 时钟信号由内部RC振荡电路提供，时钟频率一般为8 kHz，这个信号一般用于独立看门狗时钟。

HSE（外部高速时钟）: 时钟信号由外部晶振提供，时钟频率一般在4-16MHz，是经常会用到的时钟源。

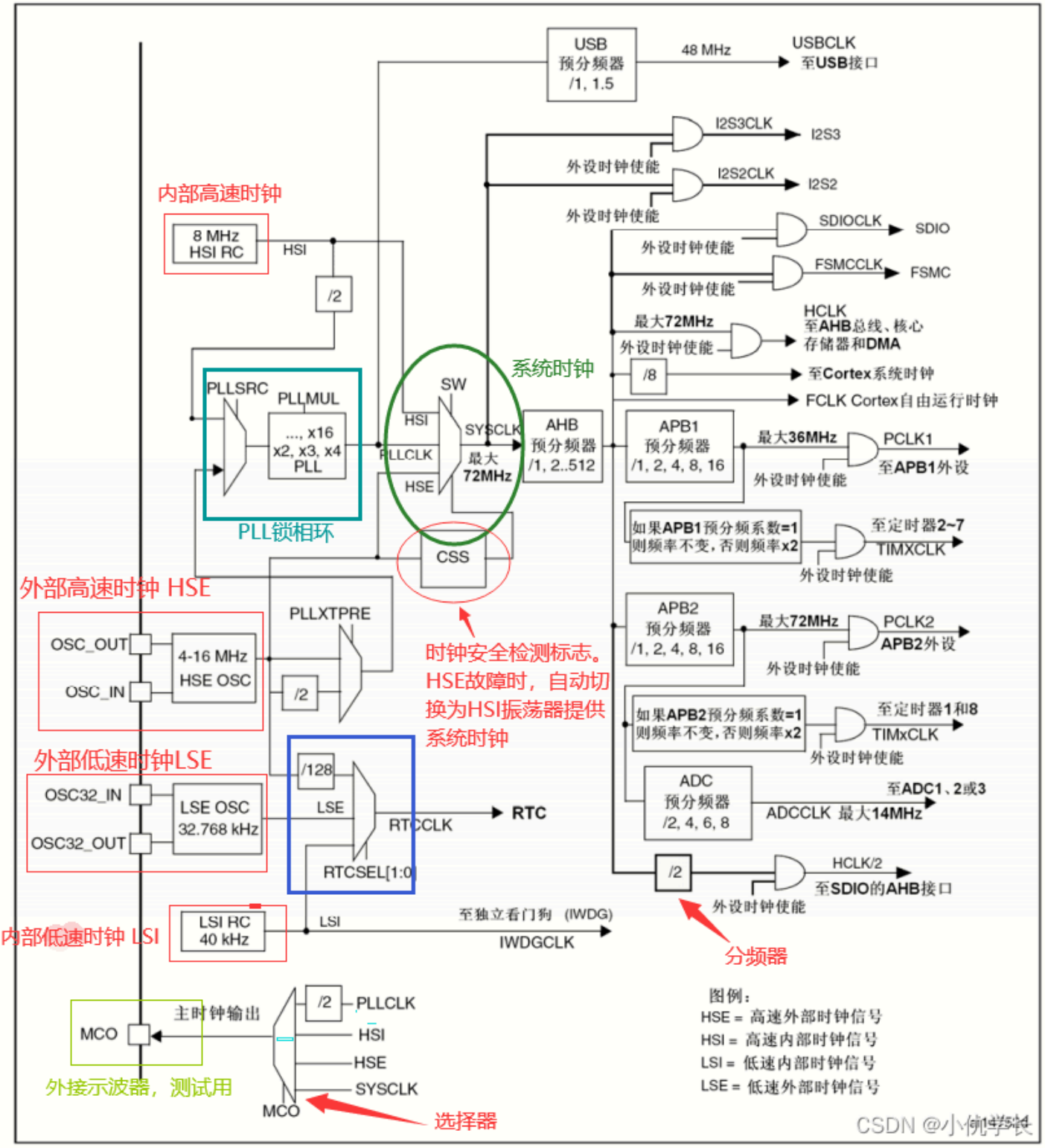
LSE（外部低速时钟）: 时钟信号由外部晶振提供，时钟频率一般为32.768KHz，这个信号一般用于RTC实时时钟。

下图是时钟树的结构图，看着先不用害怕，可以先看个大概，看看图中标记的部分

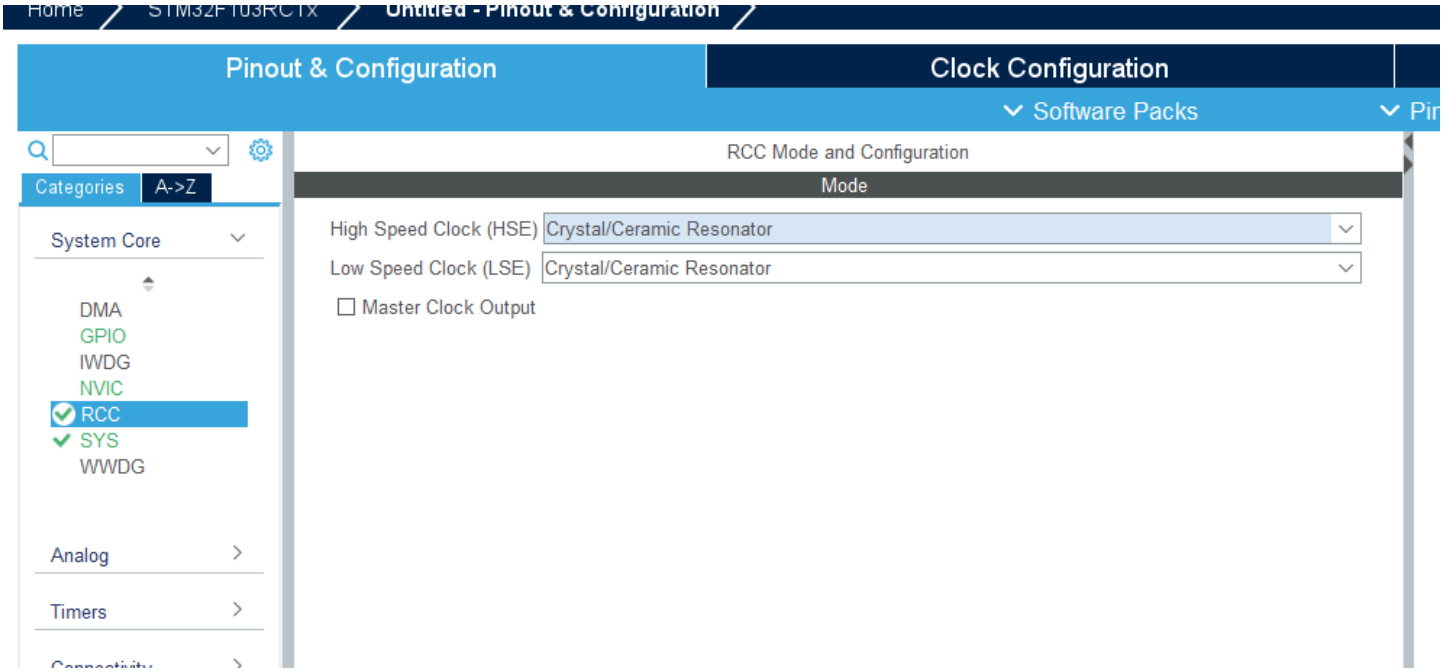
更详细的可以看这个链接https://www.bilibili.com/video/BV1ph4y1e7Ey/?spm_id_from=333.1387.upload.video_card.click

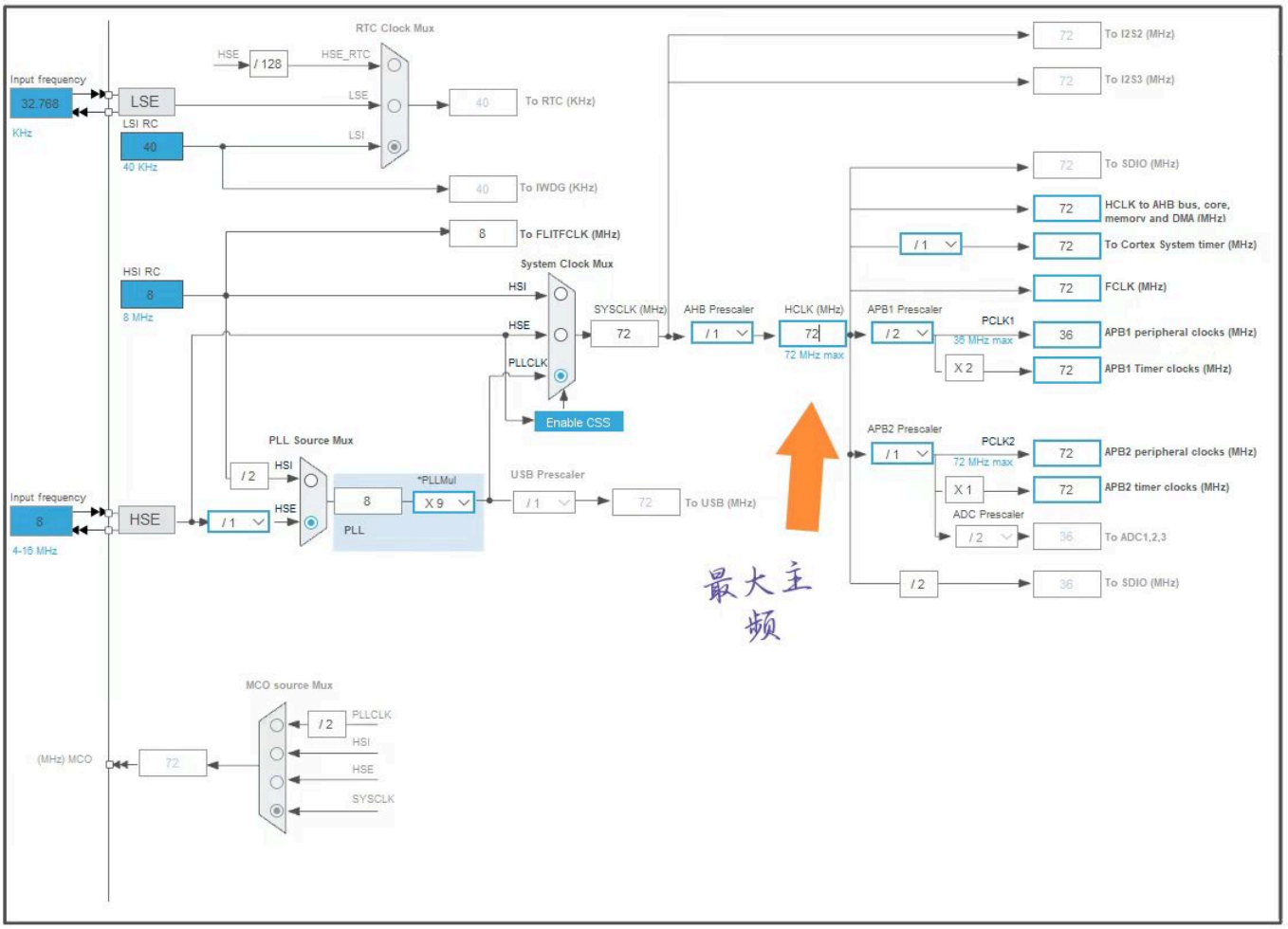
第二个图是cubumx上的时钟树设置图

图8　　时钟树



正常情况下，我们一般设置主频到最大值，有时可能会发现不让设置到显示的最大值，此时是未选择外部高速时钟，后面会讲到，就是下面这种图HSE选择外部高速晶振





2、定时器（TIM）

定时器就是计数器，定时器的计数速度由时钟频率（上文介绍的时钟频率）直接决定

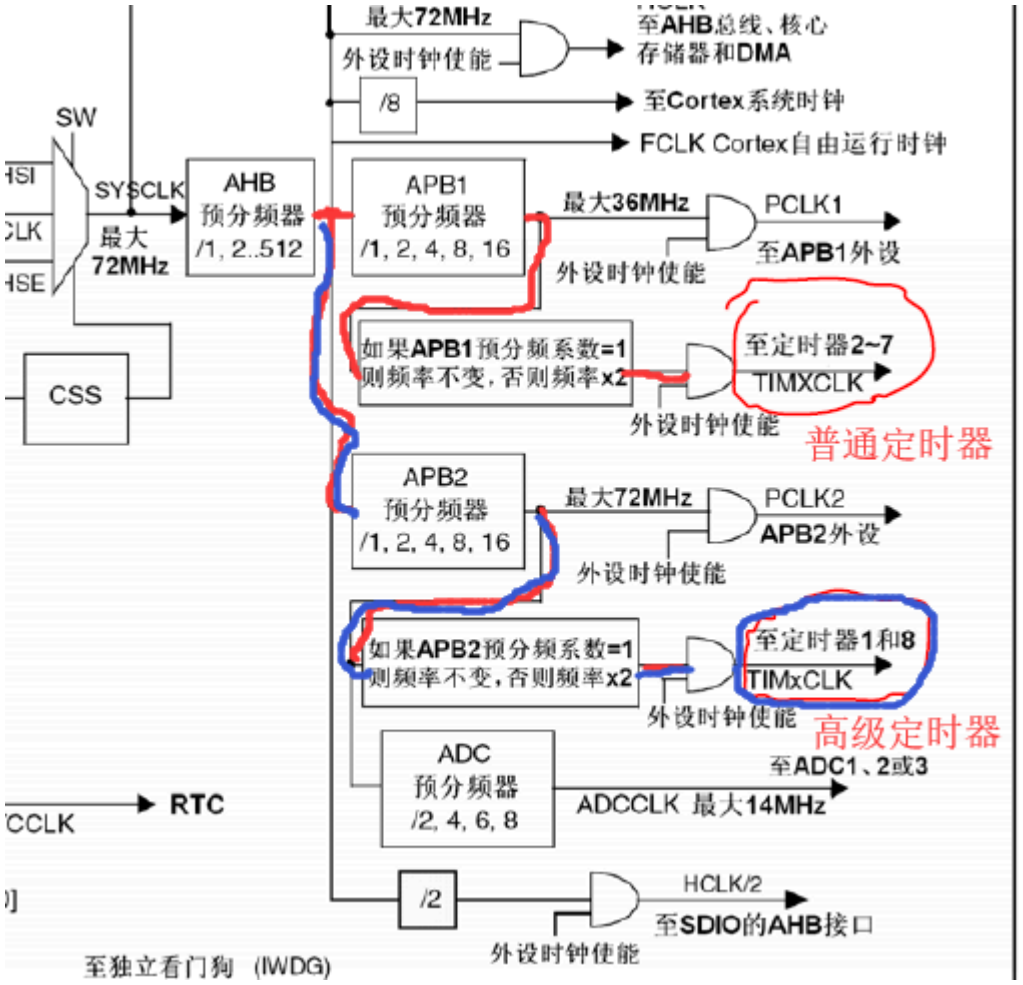
为什么说是计数器？

- 计数器也是单片机当中的一个内部外设，计数器顾名思义是用来计数的，就和我们的秒表一样，秒表实际上就是一个计数器，每隔一个单位走一个格（就是计一个数），因为计数器的计数时间周期是固定的，因此到了一定时间只要用计数值*计数时间周期，就能得到一个时间段，这个时间段就是我们定的时间（这就是定时器了）。计数器和定时器其实是一回事。

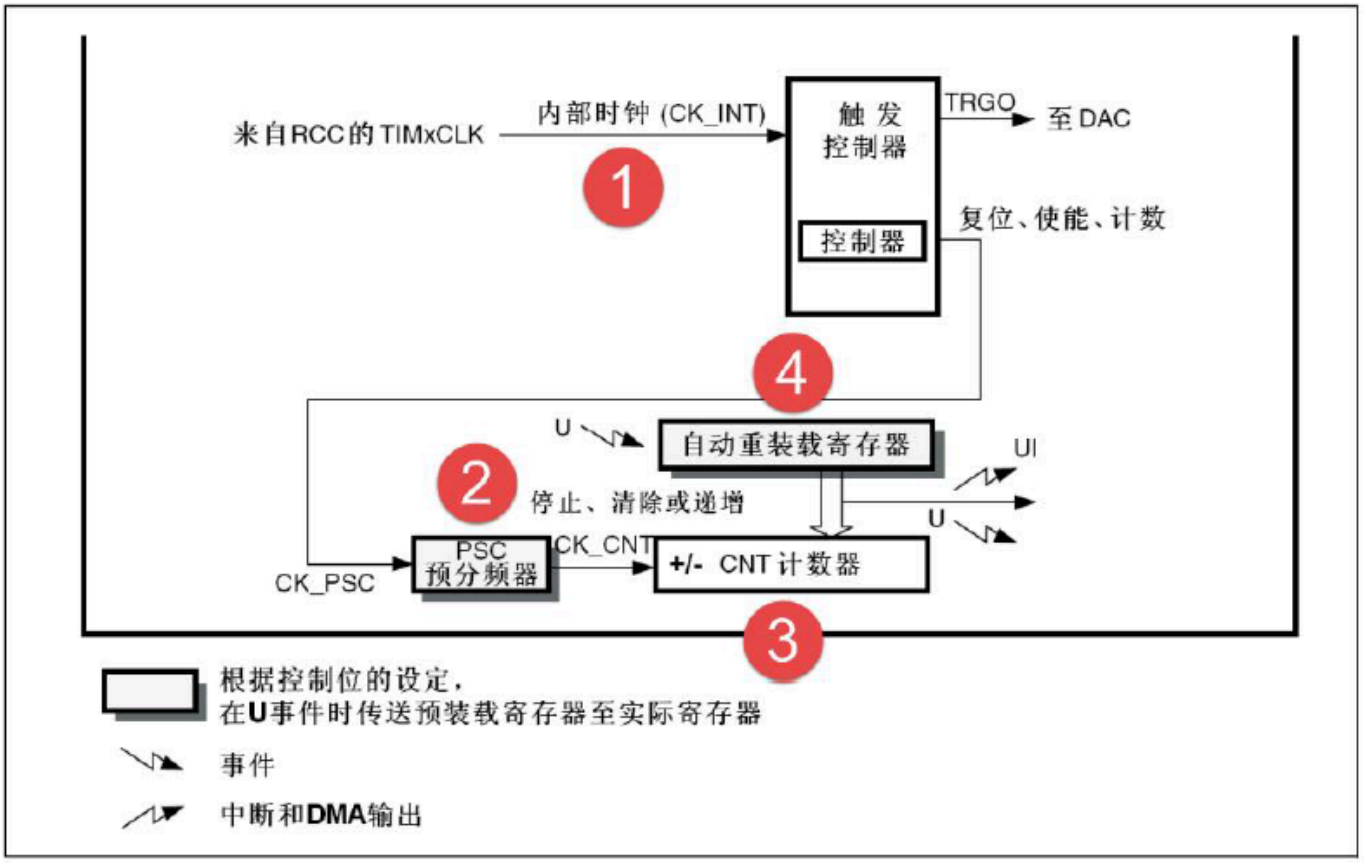
定时器有什么作用？

- 定时器可以让芯片在执行主程序的同时，可以（通过定时器）具有计时功能，到了一定时间（计时结束）后，定时器会产生中断提醒CPU，CPU会去处理中断并执行定时器的ISR（ISR（Interrupt Service Routine，中断服务程序）是一个用于处理硬件中断的特定程序。中断是硬件或软件引起的事件，会暂时打断当前正在运行的任务，以便紧急处理某个事件。ISR的目的是快速响应中断信号，执行所需的操作，并尽快返回继续执行被打断的任务。）。从而去执行预先设定好的事件。
- 定时器就好像是CPU的一个秘书一样，这个秘书专门管帮CPU来计时，并到时间后提醒CPU要做某件事情。所以CPU有了定时器之后，只需要预先把自己XX时间之后必须要做的事情绑定到定时器中断ISR即可，到了时间之后定时器就会以中断的方式提醒CPU来处理这个事情。

下图是定时器外设在时钟树中的“树枝”的位置



下图是简化的定时器的核心计数原理，后面有更详细的原理图，先简单理解



- 1、时钟源：定时器时钟 TIMxCLK，即内部时钟 CK_INT，经 APB1 预分频器后（这个看更上面的）分频提供，如果 APB1 预分频系数等于 1，则频率不变，否则频率乘以 2，库函数中 APB1 预分频的系数是 2，即 PCLK1=36M，所以定时器时钟 TIMxCLK=36*2=72M。
- 2、计数器时钟：定时器时钟经过 PSC 预分频器之后，即 CK_CNT，用来驱动计数器计数。PSC 是一个 16 位的预分频器，可以对定时器时钟 TIMxCLK 进行 1~65536 之间的任何一个数进行分频。具体计算方式为：CK_CNT=TIMxCLK/(PSC+1)。 **为什么分母加一，因为从0开始**
- 3、计数器：计数器 CNT 是一个 16 位的计数器，最大计数值为 65535。当计数达到自动重装载寄存器的时候产生更新事件，并清零从头开始计数。
- 4、自动重装载寄存器：自动重装载寄存器 ARR 是一个 16 位的寄存器，这里面装着计数器能计数的最大数值。当计数到这个值的时候，如果使能了中断的话，定时器就产生中断。
- 5、**定时时间的计算**：如果定时器的输入时钟频率为2 MHz，预分频器值为1999，计数器重载值为999，那么定时器溢出的时间计算如下：Timeout = （（1999 + 1） * （999 + 1） ） / 2; Timeout = 1000000us = 1s。这意味着定时器每1秒触发一次中断。

1）定时器的分类与核心特性

STM32总共有8个定时器，分别是2个高级定时器（TIM1、TIM8），4个通用定时器（TIM2、TIM3、TIM4、TIM5）和2个基本定时器（TIM5、TIM6）

定时器种类	位数	计数器模式	产生DMA请求	捕获/比较通道	互补输出	特殊应用场景
高级定时器(TIM1,TIM8)	16	向上，向下，向上/下	可以	4	有	带死区控制盒紧急刹车，可应用于PWM电机控制
通用定时器(TIM2~TIM5)	16	向上，向下，向上/下	可以	4	无	通用。定时计数，PWM输出，输入捕获，输出比较
基本定时器(TIM6,TIM7)	16	向上，向下，向上/下	可以	0	无	主要应用于驱动DAC

CSDN @KevinFlyn

STM32的多定时器中我们使用最多的是高级定时器和通用定时器，而高级定时器一般也是用作通用定时器的功能，下面我们就以通用定时器为例进行讲解，其功能和特点包括：

- 位于低速的APB1总线上(APB1)
- 16 位向上、向下、向上/向下(中心对齐)计数模式，自动装载计数器（TIMx_CNT）。
- 16 位可编程(可以实时修改)预分频器(TIMx_PSC)，计数器时钟频率的分频系数 为 1～65535 之间的任意数值。
- 4 个独立通道（TIMx_CH1~4），这些通道可以用来作为：

  ① 输入捕获

  ② 输出比较

  ③ PWM 生成(边缘或中间对齐模式)

  ④ 单脉冲模式输出

详述这些模式的原理和作用

输入捕获

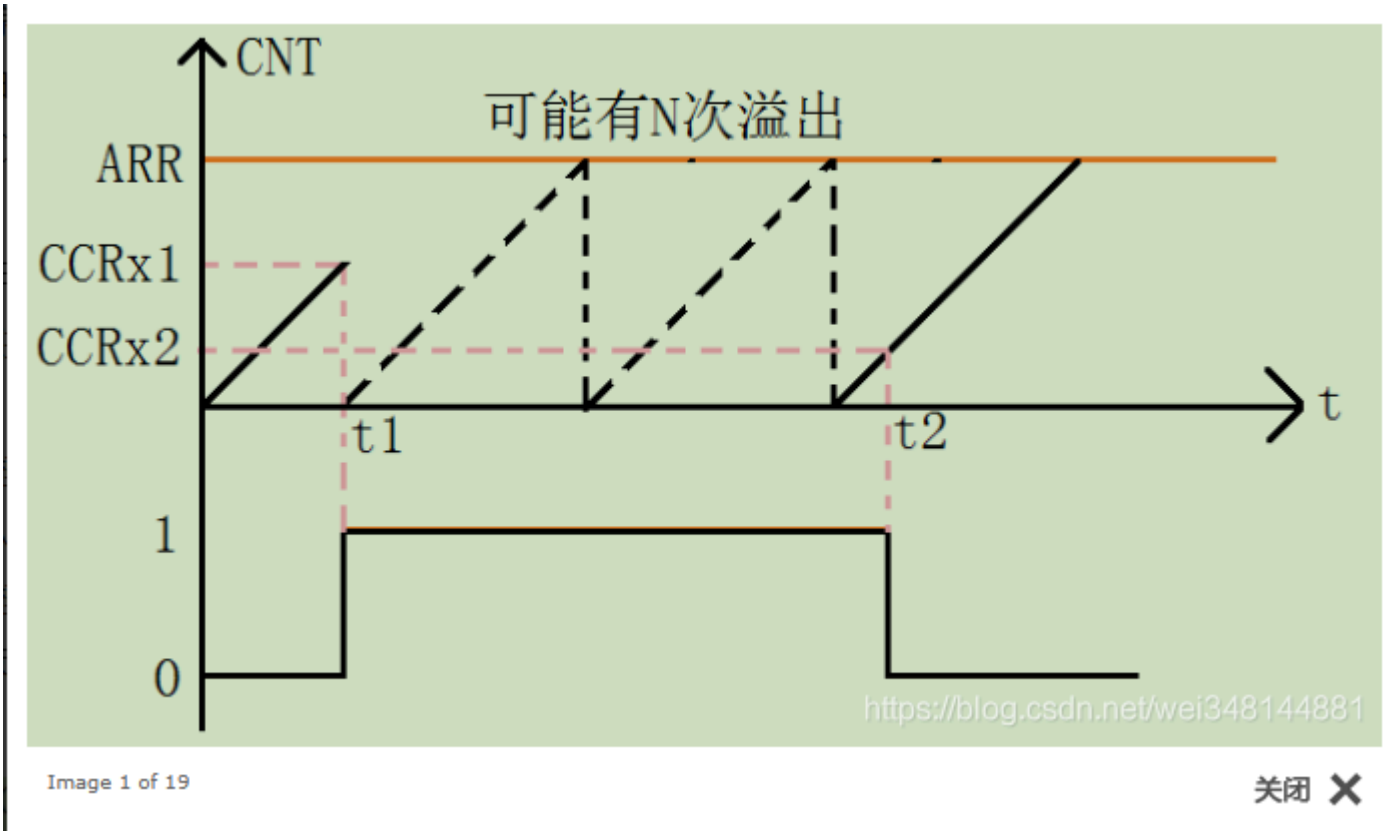
1、什么是捕获

捕获就是通过检测捕获通道上的边沿信号。在边沿信号发生跳变（比如上升沿/下降沿）的时候，将当前定时器的值(TIMx_CNT)存放到对应的通道的捕获/比较寄存器(TIMx_CCR)里面，完成一次捕获。

2、输入捕获模式能干什么

输入捕获模式可以用来测量脉冲宽度或者测量频率。

下图是输入捕获测量高电平脉宽的原理，假定定时器工作在向上计数模式，图中 t1~t2 时间，就是需要测量的高电平时间。



测量方法如下：首先设置定时器通道 x 为上升沿捕获，这样，t1 时刻，就会捕获到当前的 CNT 值，然后立即清零 CNT，并设置通道 x为下降沿捕获，这样到 t2 时刻，又会发生捕获事件，得到此时的 CNT 值，记为 CCRx2。这样，根据定时器的计数频率，我们就可以算出 t1~t2 的时间，从而得到高电平脉宽。

输出比较

1、什么是比较

比较就是计数器的值和与装载的值进行比较。可以设定一个阈值。

2、输出比较模式原理

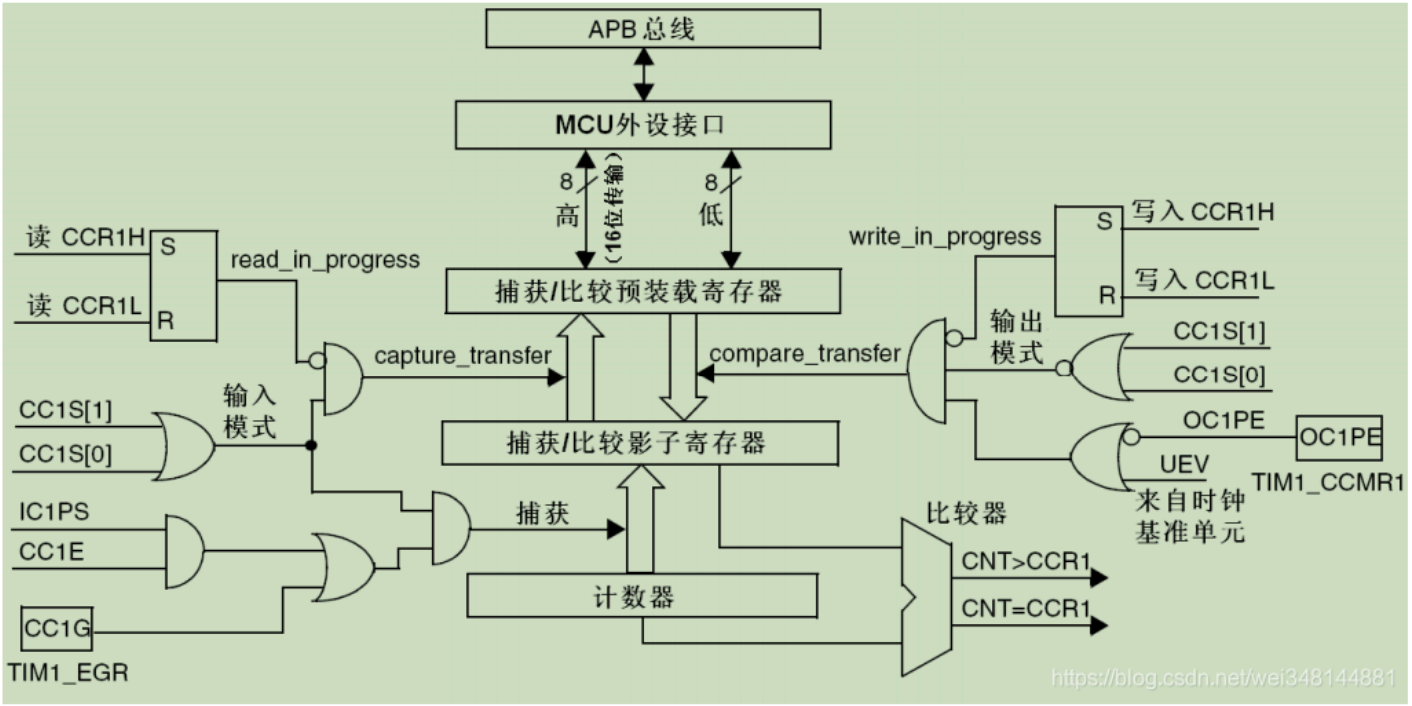


Image 4 of 19

关闭

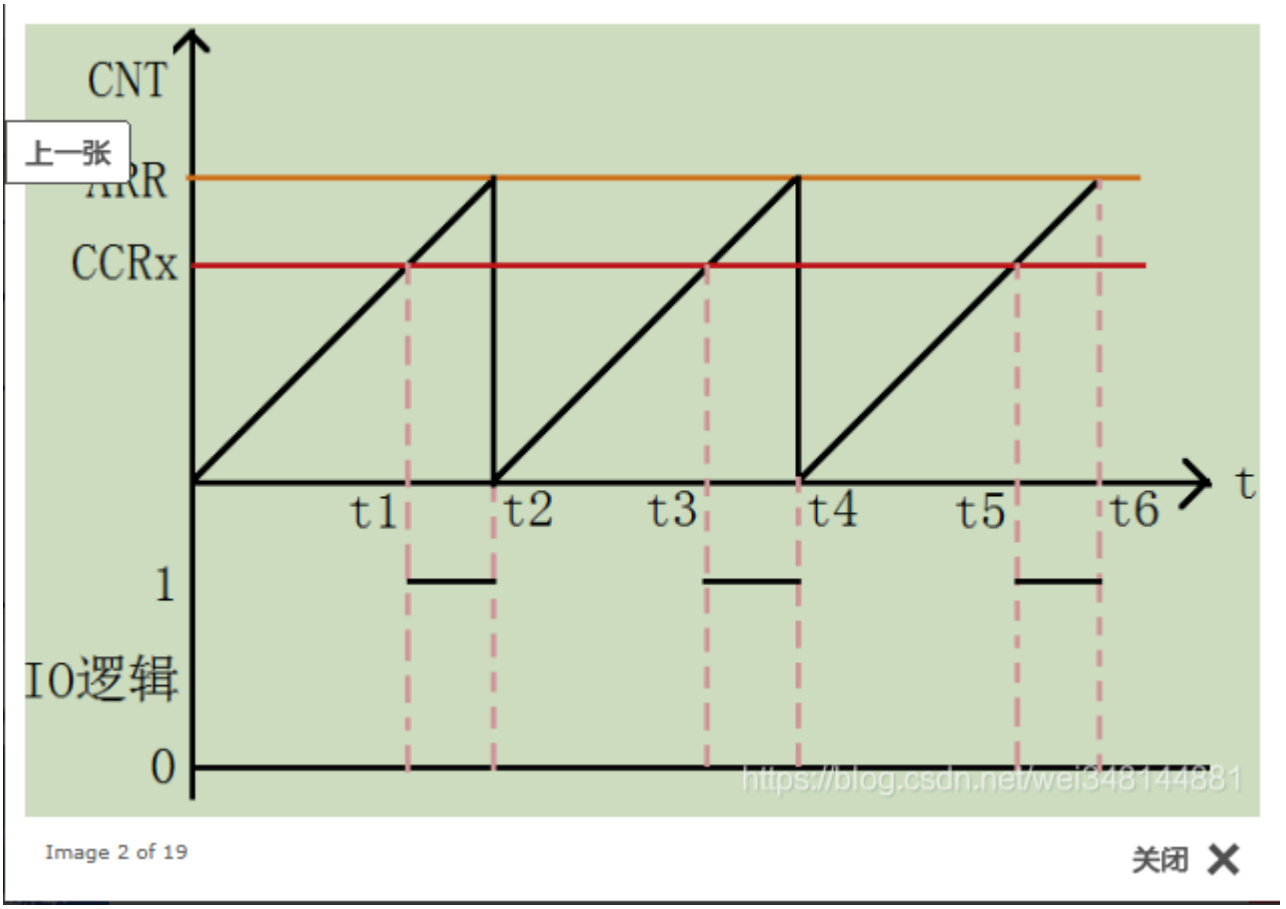
在捕获模式下，捕获发生在影子寄存器上，然后再复制到预装载寄存器中。

在比较模式下，预装载寄存器的内容被复制到影子寄存器中，然后影子寄存器的内容和计数器进行比较。

一句话总结：在输出比较模式下，预装载寄存器的内容会先被复制到影子寄存器中，以此作为比较的基准阈值。随后，定时器计数器的实时计数值与影子寄存器中的基准值进行比对，当两者相等时即产生匹配事件，从而控制输出电平翻转或触发相应中断。

PWM 生成(边缘或中间对齐模式)

在这个模式下，定时器外设输出PWM波，其原理可以结合输出比较模式看



假定定时器工作在向上计数 PWM模式，且当 CNT=CCRx 时输出 1。当 CNT 达到 ARR 值的时候，重新归零，然后重新向上计数，依次循环。改变 CCRx 的值，就可以改变 PWM 输出的占空比，改变 ARR 的值，就可以改变 PWM 输出的频率。**后续会更详细介绍。**

单脉冲输出模式

计数器启动后，在更新事件来临之前的时间段内实现一定个数的脉冲输出，当发生更新事件时计数器停止计数，导致后续不再有变化的PWM波形输出。输出的脉冲个数可以一个或几个。如果是通用计数器就是1个，如果是高级定时器，脉冲个数与RCR数值和计数模式二者有关。用到的比较少，不过多介绍了。

- 可使用外部信号（TIMx_ETR）控制定时器和定时器互连（可以用 1 个定时器控制另外一个定时器）的同步电路。
- 如下事件发生时产生中断/DMA：

  ①更新：计数器向上溢出/向下溢出，计数器初始化(通过软件或者内部/外部触发)

  ②触发事件(计数器启动、停止、初始化或者由内部/外部触发计数)

  ③输入捕获

  ④输出比较

  ⑤支持针对定位的增量(正交)编码器和霍尔传感器电路

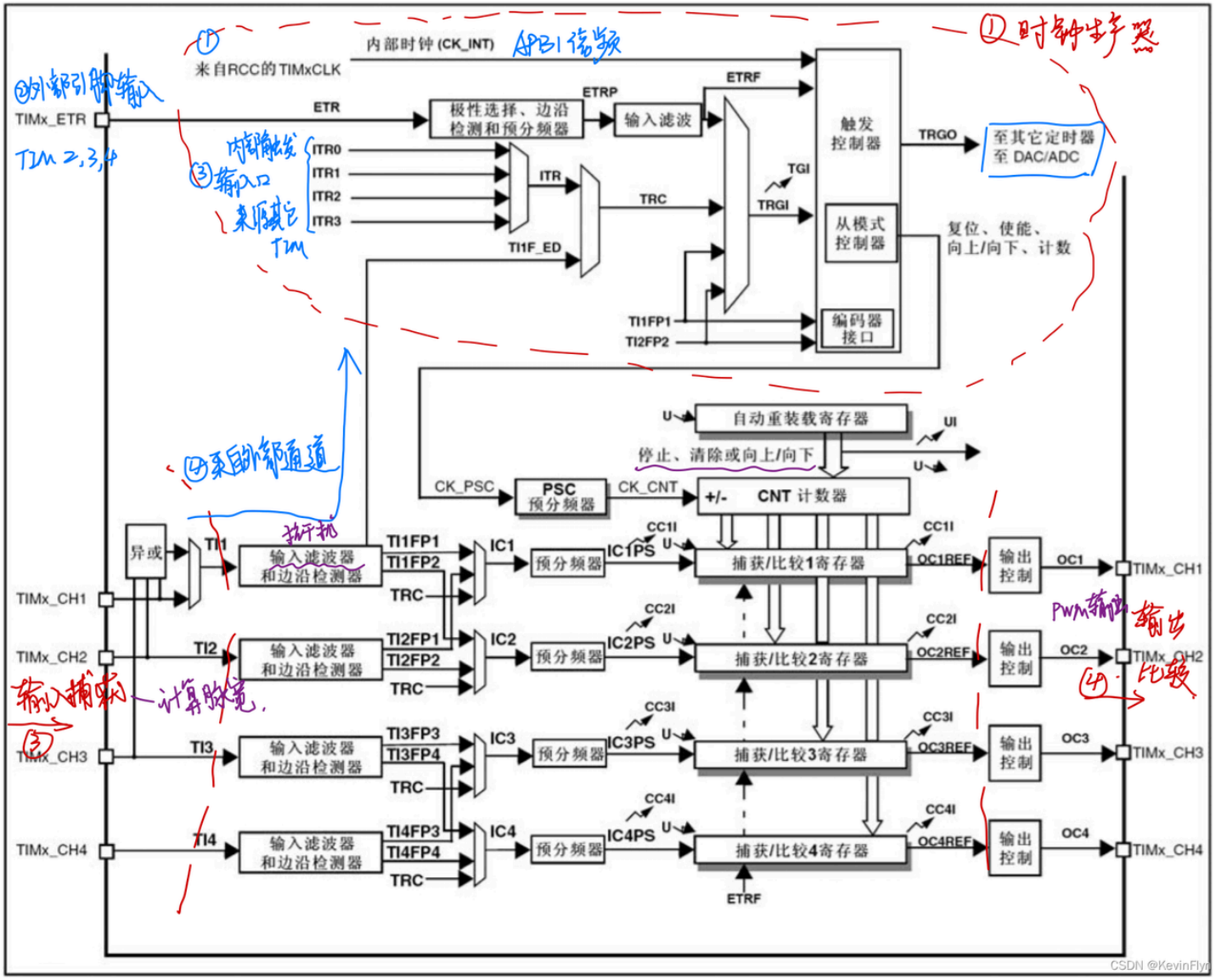
  ⑥触发输入作为外部时钟或者按周期的电流管理

- STM32 的通用定时器可以被用于：测量输入信号的脉冲长度(输入捕获)或者产生输出波形(输出比较和 PWM)等。
- 使用定时器预分频器和 RCC 时钟控制器预分频器，脉冲长度和波形周期可以在几个微秒到几个毫秒间调整。

2）定时器原理

这是定时器的工作框图，比较难看懂，想更清楚的了解可以看这个链接
https://www.bilibili.com/video/BV11u4y1A7gS/?spm_id_from=333.337.search-card.all.click&vd_source=3177b054e170ae41b361ede38c72f8fc

如果希望更深入的了解，从寄存器的角度理解，可以直接看STM32F1的文档



大致说一下上面的图：

- （用红色笔表示出）①时钟产生器部分，②时基单元部分，③输入捕获部分、④输出比较部分。
- （用蓝色笔表示出）

  ①内部时钟(CK_INT)

②外部时钟模式：外部触发输入(ETR)

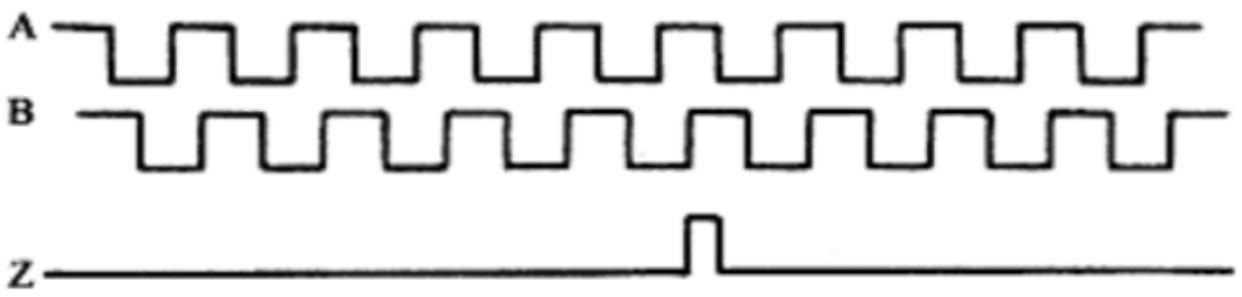
③内部触发输入(ITRx)：使用一个定时器作为另一个定时器的预分频器，如可以配置一个定时器Timer1而作为另一个定时器Timer2的预分频器。

④外部时钟模式：外部输入脚(TIx)

3）定时器的编码器模式

编码器是一种将机械运动（如旋转或线性位移）转换为电信号的装置，通过这些信号可以测量运动的位置、速度或方向。STM32的定时器编码器模式是一种硬件功能，专门用于读取增量式编码器的输出信号，实现对电机等设备的位置、速度和方向的精确测量。对于RM比赛不可或缺的电机控制，编码器的使用助于我们更精准的控制电机的运动，同时校内赛也需要进行一定的电机控制。

- 编码器原理：增量式编码器有两个脉冲输出，A相和B相，并且两个相位永远存在90°相位差。 如果两个信号相位差为90度，则这两个信号称为正交。由于两个信号相差90度，因此可以根据两个信号哪个先哪个后来判断方向、并且可以根据AB相脉冲信号数量测得速度，位移等，对于电机控制比较方便。
- 编码器信号：



A 脉冲输出

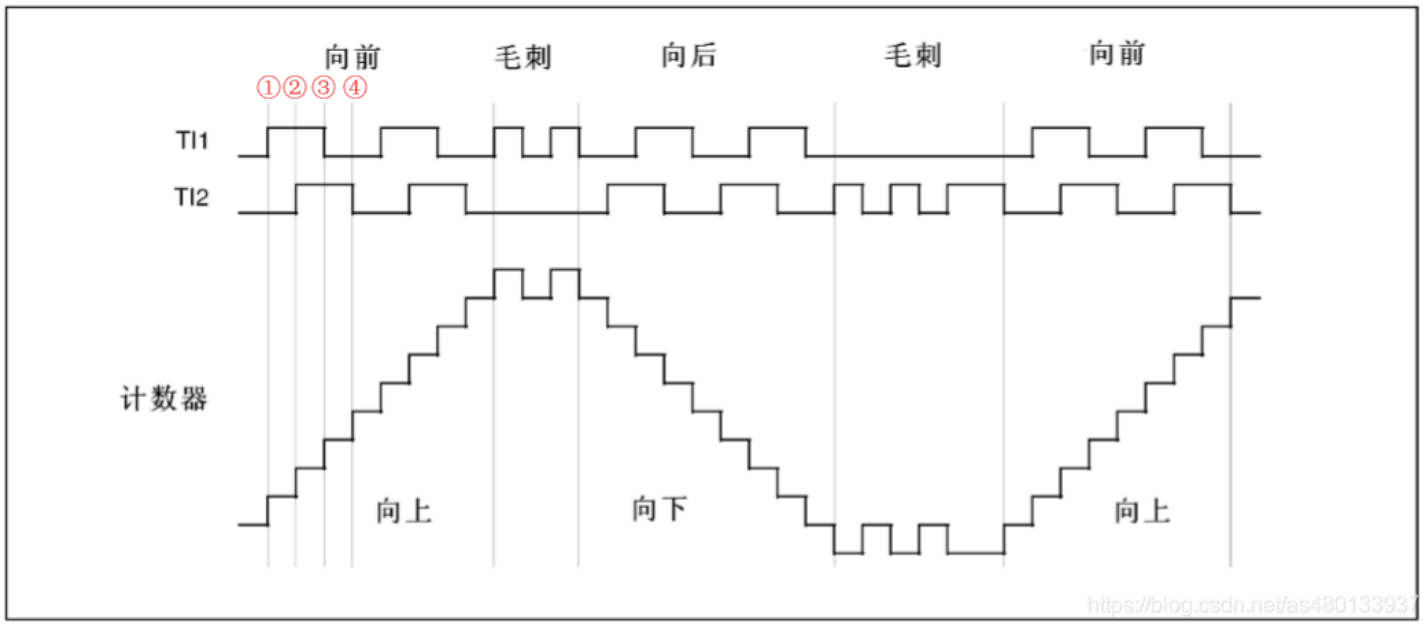
B 脉冲输出

Z 零点信号 当编码器旋转到零点时，Z信号会发出一个脉冲表示现在是零位置 表示编码器转了1圈，可用来记录编码器转了多少圈，从而知道运行距离

VCC 电源线

GND 地线

- 在TL1和TL2都计数(A相和B相都计数)



1时刻：TI2为低电平，TI1上升沿跳变，计数器向上计数；

2时刻：TI1为高电平，TI2上升沿跳变，计数器仍然向上计数；

3时刻：TI2为高电平，TI1下降沿跳变，计数器仍然向上计数；

4时刻：TI1为低电平，TI2下降沿跳变，计数器仍然向上计数。

3、PWM

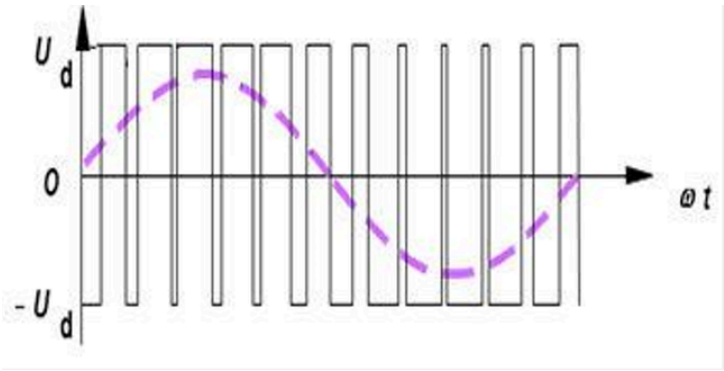
1）什么是PWM

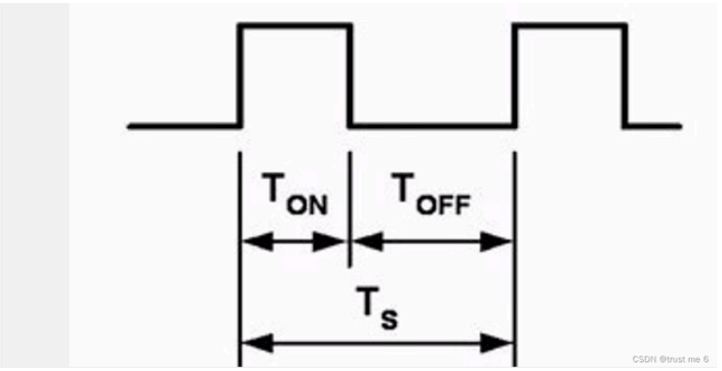
脉冲宽度调制(PWM)，是英文“Pulse Width Modulation”的缩写，简称脉宽调制，可以理解为控制脉冲的宽度，利用微处理器的数字输出来对模拟电路进行控制的一种非常有效的技术，它通过控制信号的脉冲宽度，实现对电压或电流的精确控制。在嵌入式的应用中，如果数字信号控制的LED只有完全亮与灭两种状态，怎么能实现控制亮度大小呢？这就用到了PWM等效输出模拟量。

2）PWM的相关参数

PWM主要有三个参数：频率、占空比、分辨率

- 频率（Frequency）：是指1秒钟内信号从高电平到低电平再回到高电平的次数(一个周期)，也就是说一秒钟PWM有多少个周期。
- 占空比（Duty Cycle）：是一个脉冲周期内，高电平的时间与整个周期时间的比例，占空比决定了PWM等效出来的模拟电压的大小，占空比越大，模拟电压越趋近于高电平。
- 分辨率（Resolution）：就是占空比变化的快慢，占空比变化的细腻程度。占空比跳的快如按照1%跳变与按照0.1%跳变，那么0.1%的跳变就越细腻，越柔和。





频率 = 1 / TS

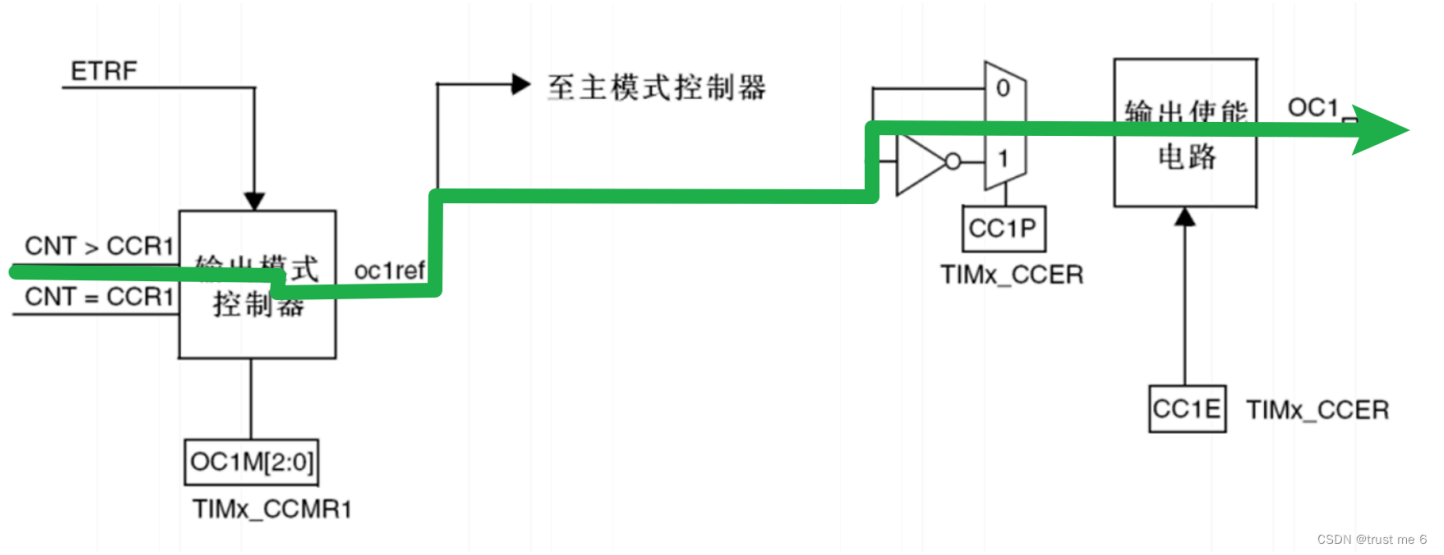
占空比 = TON / TS

分辨率 = 占空比变化步距

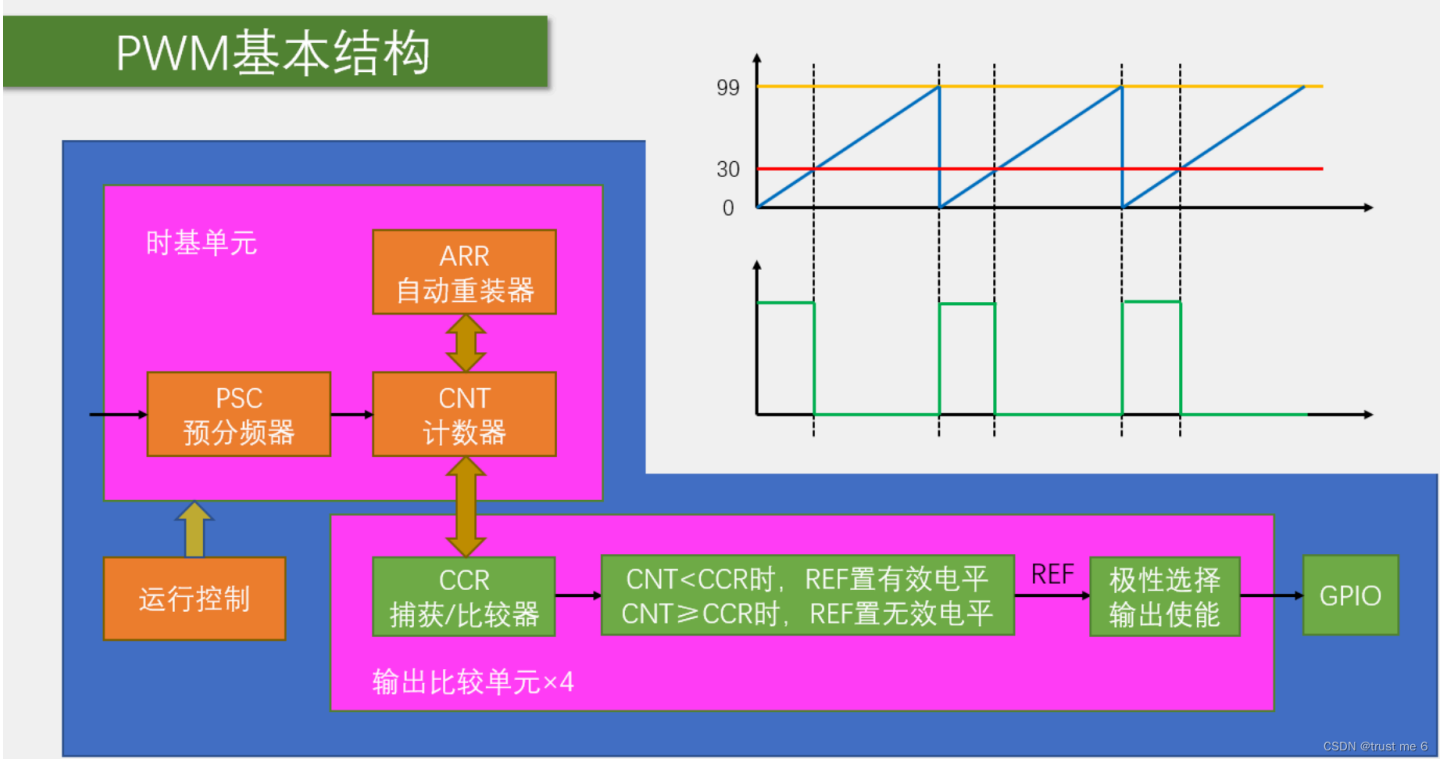
利用PWM调节LED亮度，就是让LED不断地点亮、熄灭、点亮、熄灭...，当这个点亮熄灭的频率足够大时，LED就不会闪烁了，而是呈现出一个中等的亮度。**调控点亮、熄灭的比例**时，就能让LED呈现出不同的亮度级别。这也是PWM的基本思想。

3）定时器怎么输出波形

- 第一步：CNT（计数器）与CCR（捕获比较寄存器）（上述定时器的内容中有提到），如果CNT大于or等于CCR，就会给输出模式控制器传一个信号；
- 第二步：输出模式控制器就会改变它输出OC1REF的高低电平，REF信号实际上就是信号的高低电平（REF:reference，参考信号）；
- 第三步：CC1P：极性选择寄存器，给0，信号就会走对应的0那一路，信号不变；给1，信号会经过一个非门，极性翻转；
- 第四步：信号从OC1输出。



4)PWM的基本结构和计算公式



PWM频率： Freq = CK_PSC / (PSC + 1) / (ARR + 1) （CK_PSC见前文）

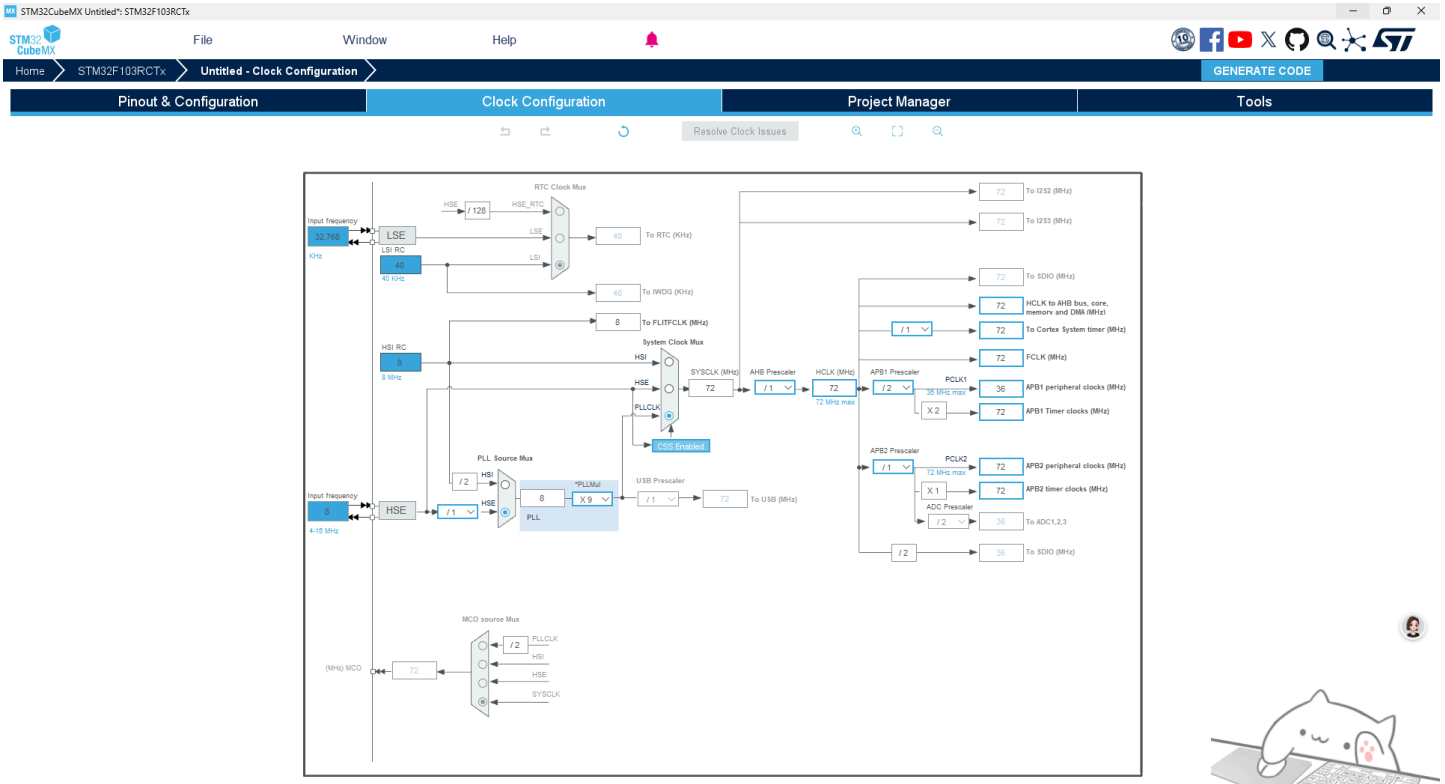
PWM占空比： Duty = CCR / (ARR + 1)

PWM基本原理比较好理解，更重要的是理解怎么产生的，然后实质还是要理解定时器的原理

二、结合STM32CubeMX理解

本文主要讲述TIM相关的配置，其他相关配置前序内容已经完成

1、时钟树

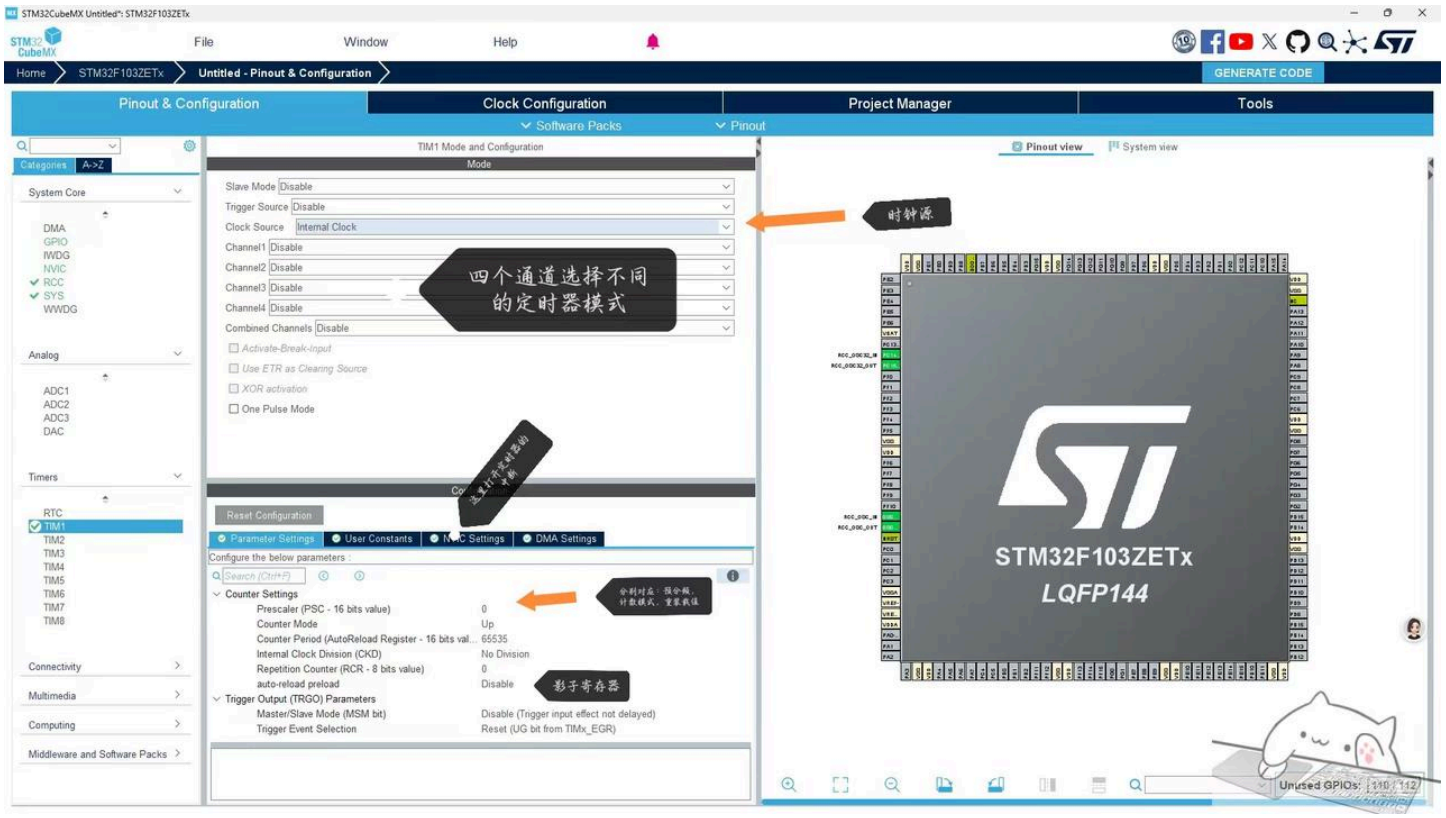




- 高级定时器（TIM1、TIM8）：挂载在APB2 总线上。这类定时器功能最丰富，支持互补输出、死区控制等，适用于电机控制等场景。
- 通用定时器（TIM2-TIM5）：挂载在APB1 总线上。它们是应用最广泛的定时器，支持定时计数、输入捕获、输出比较等基础功能。
- 基本定时器（TIM6、TIM7）：挂载在APB1 总线上。功能相对简单，主要用于定时和触发 DAC 转换。

APB总线对应于上图中靠右的位置

2、定时器



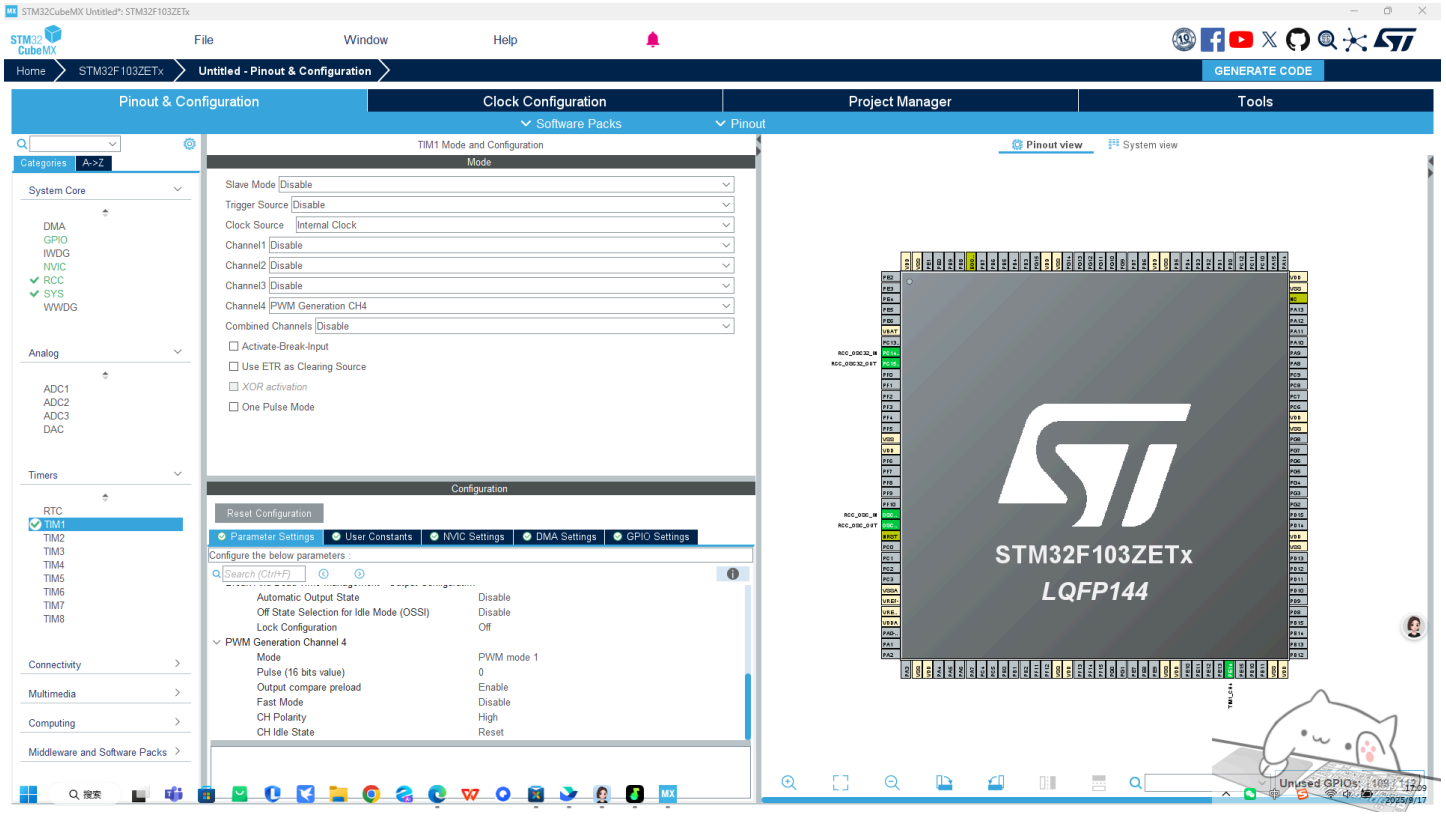
- Slave Mode（从模式）：用于设置定时器是否工作在 “从模式” 下。当选择非 “Disable（禁用）” 的模式时，定时器不再完全由自身内部时钟或软件触发来控制计数等操作，而是会被外部或其他内部信号（触发源）所同步或控制，比如可以让定时器的启动、停止、复位等动作由外部引脚输入的信号，或者其他定时器、外设的输出信号来触发。
- Trigger Source（触发源）：当 “Slave Mode” 选择了需要触发源的模式（如触发模式、复位模式等）时，这个选项用于指定究竟是哪个信号作为触发源来控制定时器的从模式行为。例如，可以选择外部引脚的输入信号、其他定时器的更新事件、比较输出等作为触发源。

前两个了解即可，平常用的比较少。

其中高级定时器的时钟源可以选择ETR2及外部时钟源，我们正常选择内部时钟源即可。下面四个通道的选项及上述所说

最后一个Combined Channels通道可以选择编码器模式

其中标识为常用的几个配置，更详细的可以查找数据手册



最下方PWM配置选项中，第一个为模式设置，默认模式一为计数器小于比较寄存器的值的时候为有效电平；

第二个为比较寄存器的值，上文已经讲述其作用；第三个为影子寄存器，同理上文；第四个不过多介绍，为开启高速PWM；最后一个为极性翻转。

同样的可以在NVIC Settings中设置中断，在DMA中配置相应DMA

三、相应HAL库函数介绍

1、定时器基本模式启动函数HAL_TIM_Base_Start

全写：HAL_StatusTypeDef HAL_TIM_Base_Start(TIM_HandleTypeDef *htim);

功能： 用于启动定时器基本模式

运行流程： 当使用了HAL_TIM_Base_Start函数后，定时器将开始计数，并在达到预设的计数值时就会重新开始计数

2、定时器基本模式中断启动函数HAL_TIM_Base_Start_IT

全写： HAL_StatusTypeDef HAL_TIM_Base_Start_IT(TIM_HandleTypeDef *htim);

功能： 用于启动定时器的基本模式并使能定时器中断

运行流程： 就是当使用这个功能来启动定时器的时候，和上面的定时器基本模式启动函数不同 的是，开始计数后，在达到预设的计数值时会触发中断，进入中断处理函数，在中断处理函数中根据不同的中断源来选择回调函数，在回调函数中进一步处理

3、定时器周期到达回调函数HAL_TIM_PeriodElapsedCallback

全写： void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim);

功能： 定时器中断的回调函数。

运行流程： 当中断源是定时器周期到达的情况下，在中断处理函数中会据此进入到定时器周期到达回调函数（这个函数HAL_TIM_PeriodElapsedCallback）中进行进一步的操作

4、定时器中断处理函数HAL_TIM_IRQHandler

全写：void HAL_TIM_IRQHandler(TIM_HandleTypeDef *htim)

功能： 此功能处理TIMx全局中断

运行流程： 进入中断处理函数，在中断处理函数中根据不同的中断源来选择回调函数

5、开启编码器模式HAL_TIM_Encoder_Start;

全写：HAL_TIM_Encoder_Start(&htim2, TIM_CHANNEL_ALL);

功能：开启定时器的编码器模式



运行流程： 进入中断处理函数，在中断处理函数中根据不同的中断源来选择回调函数

首先在CubeMx中配置好之后，前面写的初始化函数就自动配置好了，可以不用管

然后在main.c文件中使用 HAL_TIM_Base_Start或者HAL_TIM_Base_Start_IT 来启动定时器（根据程序要求来）

如果是使用 HAL_TIM_Base_Start 那就去调用计数值这类数值去分析

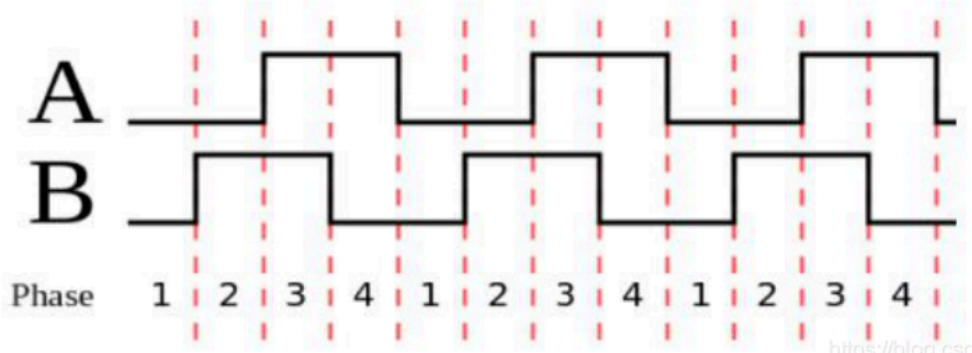
如果是使用 HAL_TIM_Base_Start_IT 那就去编写 HAL_TIM_PeriodElapsedCallback 在这个函数里面去写一些操作（建议这个函数再开一个源文件去编写，后面别的程序也可以直接复制）

四、使用PWM进行电机控制

1、AB相编码器测速

为什么要测速，获取速度，从而更好控制，很好理解

1）外部中断法



https://blog.csdn.net/weixin_44430455

捕获相	捕获情况	另一项情况	结果
A相	下降沿	B相低电平（对应图中4处）	正转
	下降沿	B相高电平（对应图中2处）	反转
A相	上升沿	B相高电平（对应图中2处）	正转
	上升沿	B相低电平（对应图中4处）	反转
B相	下降沿	A相高电平（对应图中3处）	正转
	下降沿	A相低电平（对应图中1处）	反转
B相	上升沿	A相低电平（对应图中1处）	正转
	上升沿	A相高电平（对应图中3处）	反转

我们设置两个变量用来存储捕获的脉冲数，每捕获到一次脉冲信号根据上表进行判断，正转时使其加1；反转时使其减1；

然后配置一个定时器/延迟/任务定时，每隔一段时间反馈一次测速值。

2）输入捕获法

使用编码器模式读取编码器的值

编码器模式的配置见前文，原理为利用编码器模式获取

代码实现和具体原理可见资料

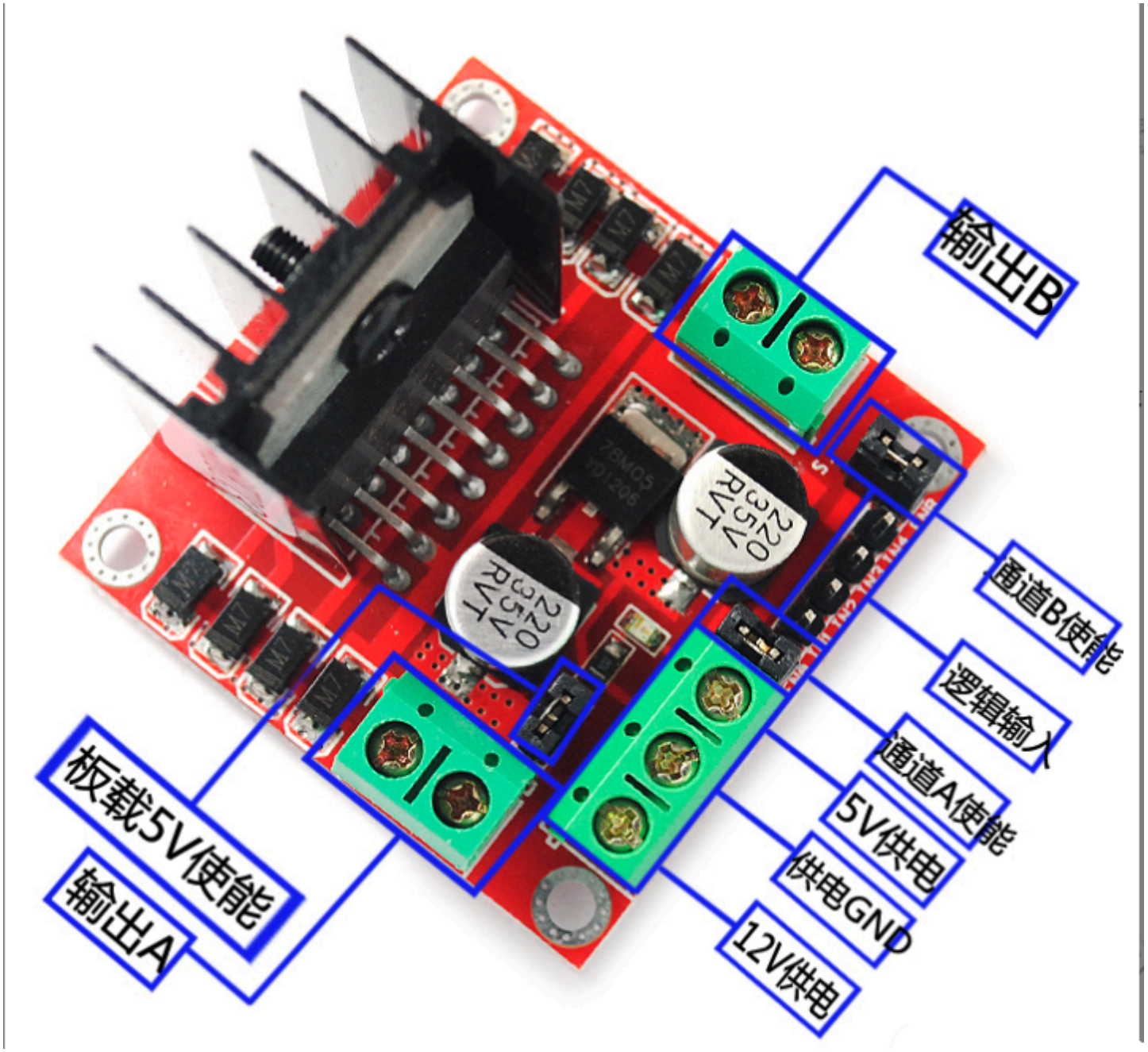
https://blog.csdn.net/yuleitao/article/details/103318237?

2、L298N电机驱动模块

电机驱动模块是电动机控制系统的重要组成部分，能够提供所需的电流和电压，确保电机的正常运行和精确控制。

驱动模块的作用

- **提供电流和电压**：电机通常需要特定的电流和电压才能正常运转，而电机驱动模块能够提供这些必要的电源，确保电机能够启动和运行。
- **控制电机的启停和转向**：驱动模块可以通过控制信号实现电机的启停、转向和转速调节等功能，从而实现对电机的精确控制。
- **保护电机**：电机驱动模块能够保护电机免受电压和电流变化的损害，延长电机的使用寿命。
- **信号转换**：驱动模块可以将微控制器发出的数字信号转换为电机可以理解的模拟信号，从而实现对电机的控制。



1）双PWM法

直流电机	旋转方式	IN1	IN2	IN3	IN4
MOTOR-A	正转(调速)	1/PWM	0		
	反转(调速)	0	1/PWM		
	待机	0	0		
	刹车	1	1		
MOTOR-B	正转(调速)			1/PWM	0
	反转(调速)			0	1/PWM
	停止			0	0
	刹车			1	1

注：“1”代表高电平；“0”代表低电平；“PWM”代表脉宽调制波，调节占空比改变转速。
IN1、IN2控制直流电机A；IN3、IN4控制直流电机B；两路是完全独立的。
输入端INx有防共态导通功能，悬空时等效于为低电平输入

CSDN @最穷不过要饭，不死总会出头

具体逻辑如上，参考资料：https://blog.csdn.net/m0_59113542/article/details/121038048

推荐PWM频率在500-1000Hz左右，较低频率控制效果不太好。

🌰 （此方法不推荐使用，多占用了一个PWM通道的同时控制效果不好）

2）单PWM+双IO口

参考资料：https://blog.csdn.net/weixin_52337250/article/details/123764689

【【模块·L298N电机驱动】不翻页！速通！电赛教程】
https://www.bilibili.com/video/BV1xKuuzkENh?vd_source=559e497a1e15d78c1c0d7052a7ada06d

PWM 调速模式			
ENA	IN1	IN2	电机状态
PWM 输出	X	X	停止
PWM 输出	0	0	制动
PWM 输出	0	1	正转
PWM 输出	1	0	反转
PWM 输出	1	1	制动

CSDN @李柿子。

非 PWM 调速模式			
ENA	IN1	IN2	电机状态
0	X	X	停止
1	0	0	制动
1	0	1	正转
1	1	0	反转
1	1	1	制动

具体逻辑如上，其实就是配置PWM输出和GPIO的输出模式，需要注意下一下pwm的周期频率，推荐在500-1000Hz左右，较低频率控制效果不太好。

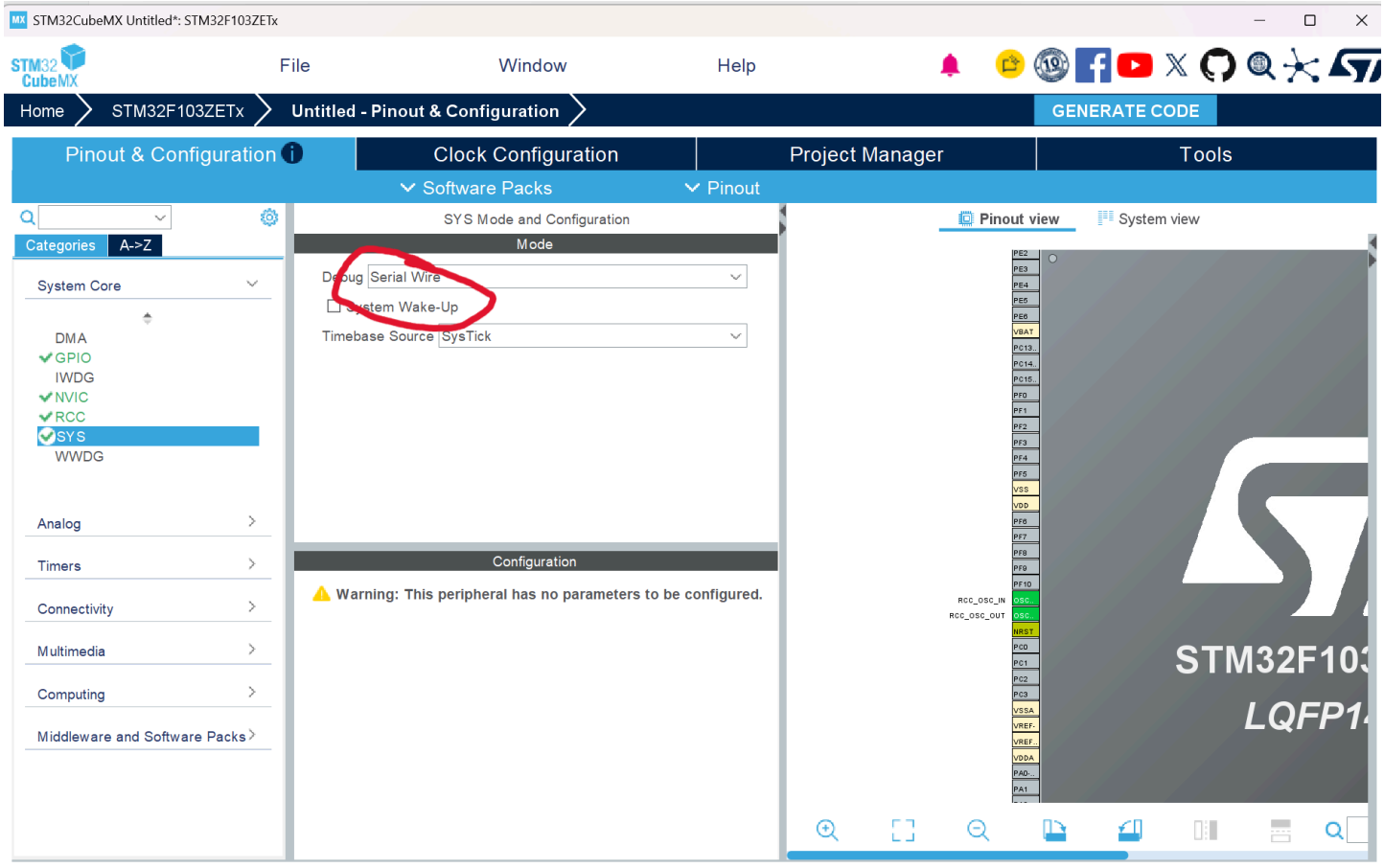
五、、实践

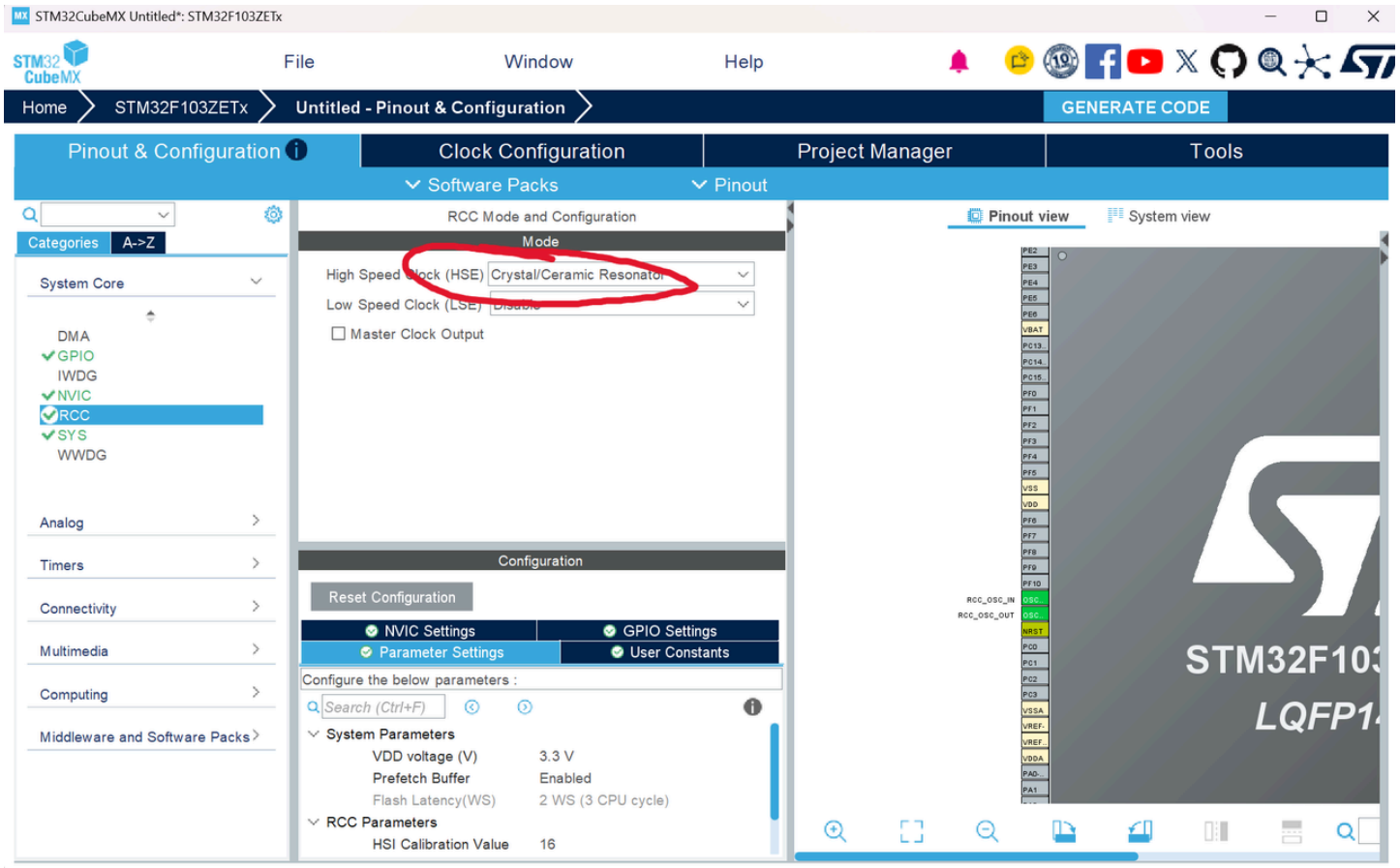
0、简单演示

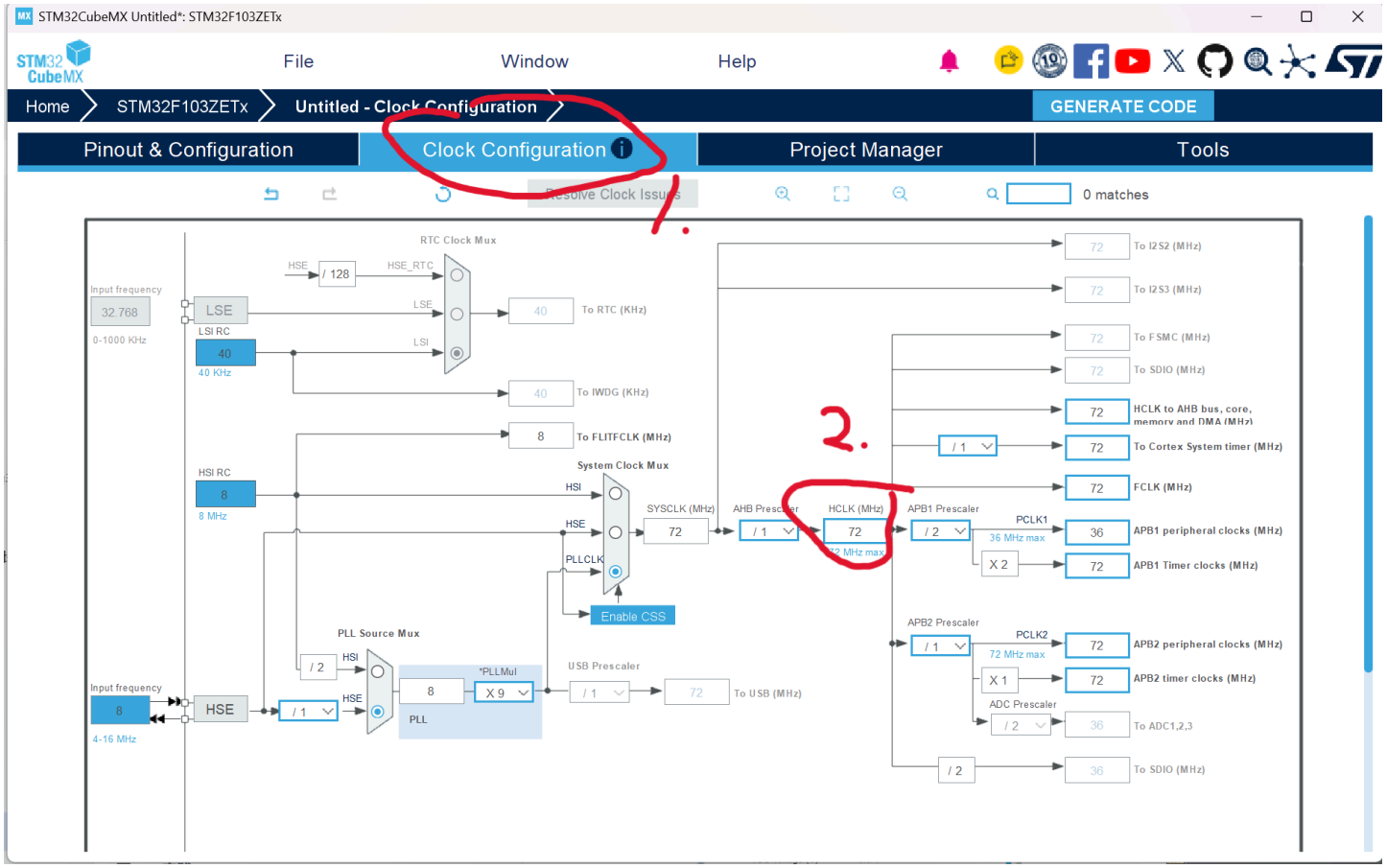
我们先用定时器的pwm功能实现一个简单的呼吸灯代码：

1.首先新建一个STM32F103ZET6的工程

起手式配置SYS为Serial Wire；配置时钟为外部晶振；时钟数主频直接设置为最大的72MHz,点击确认后，它会自动帮你配置其它的部分

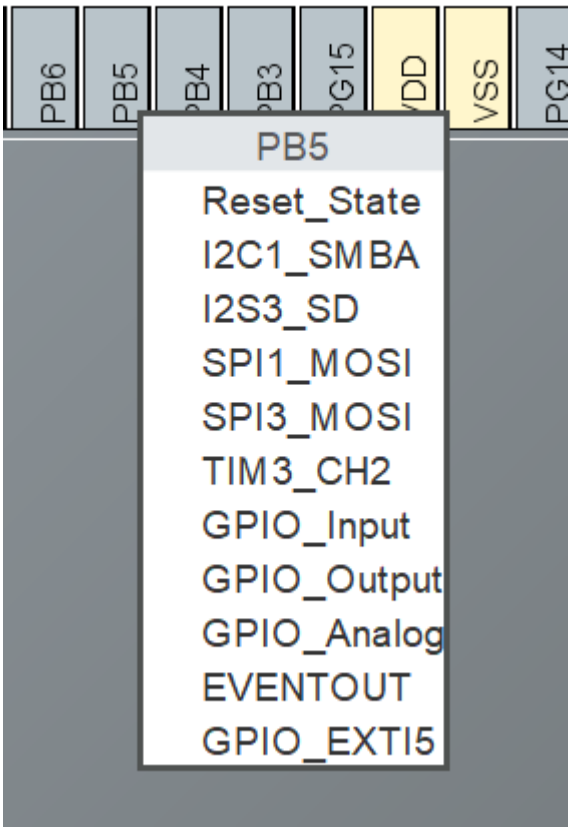






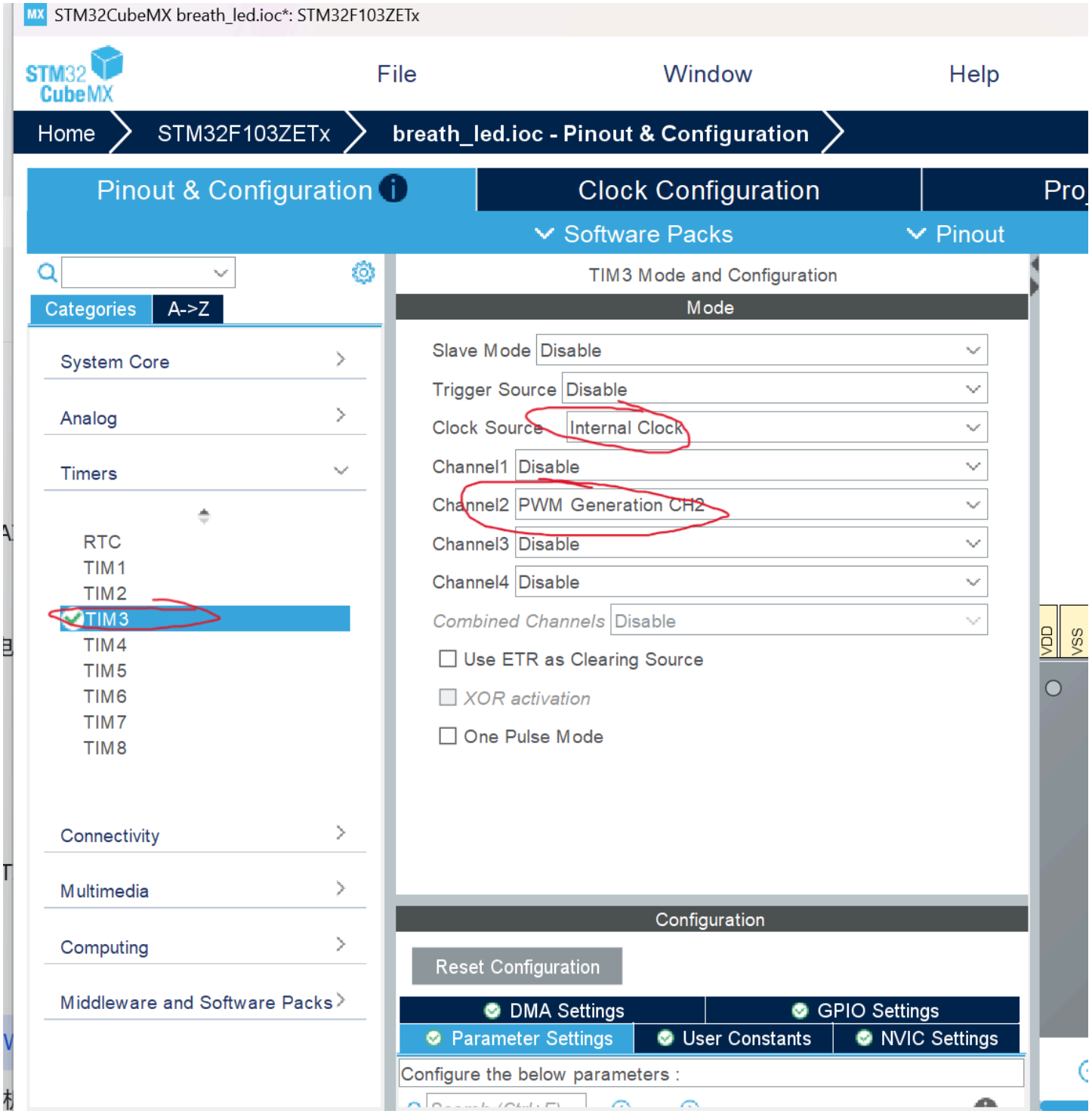
2.接下来进行TIM的配置：

我们想让板载的led实现呼吸灯，所以要为板子上的led连的接口配置为PWM输出，这里以PB5为例~~其实是因为另外一个led连着的PE5没有TIM输出~~



可以看到，PB5输出口可以接TIM3的Channel2,那么我们就去配置TIM3；

我们选中Timers的TIM3，选择时钟源为内部时钟，将Channel3配置为PWM Generation CH2



注：这里选择时钟源为内部时钟，并不是选择时钟源为内核的8MHz时钟，而是选择内核配置的时钟源，即之前配置的外部晶振

接下来我们进行TIM3的详细配置：

进入Paramater Settings,我们需要配置它的预分频器和自动重载值，假设我们想用10kHz的PWM来控制led，那么，大家可以想想这两个怎么配置，想好了再来和学长的结果对应一下~

查阅手册可得，TIM3是连接在APB2上的，那么它的基础频率即为APB2的频率，查阅刚刚配置的时钟数可以看到，APB2的频率为72MHz，那么，要想让它输出的PWM的频率为10kHz,就需要配置

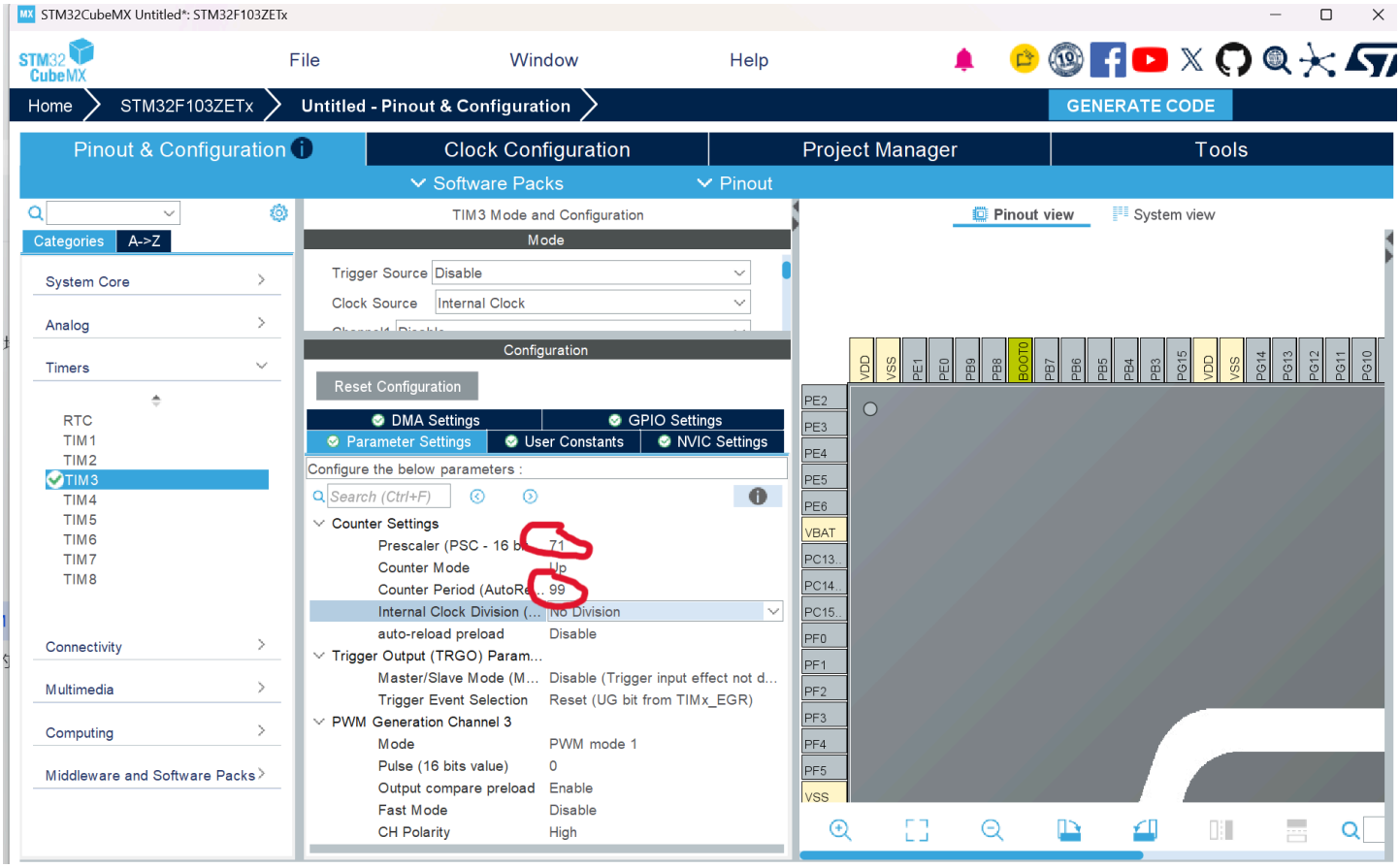
预分频器和自动重载值使得：

```
1    原始频率/（预分频器值+1）/（自动重载值+1）=目标频率
```

那么我们只需要让（预分频器值+1）*（自动重载值+1）= 72M/10k = 7200即可

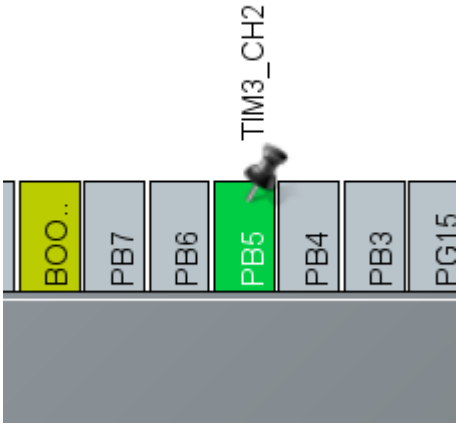
这里取预分频器值（Prescaler）=71， 自动重载值（Counter Period）= 99

其它取值也可以，只需满足上面的要求



其它的配置无需改动，感兴趣的同学可以去自行研究。

记得要把TIM3CH2的输出改到PB5上，不然PWM就连不到led上了！！！



接下来直接生成工程，用Clion打开（学长这里是用vscode，代码的部分没什么区别）

我们这里只需要用到定时器的基本模式，不会涉及到中断，所以

用定时器基本模式启动函数HAL_TIM_Base_Start

用定时器的PWM启动函数HAL_TIM_PWM_Start

然后顺便设置一个初始的比较值

```
MX_GPIO_Init();
MX_TIM3_Init();
/* USER CODE BEGIN 2 */
HAL_TIM_Base_Start(htim: &htim3);
HAL_TIM_PWM_Start(htim: &htim3, Channel: TIM_CHANNEL_2);
__HAL_TIM_SET_COMPARE(&htim3, TIM_CHANNEL_2, 0);
```

然后我们在循环中以100hz的频率控制PWM的比较值，我们希望的效果是小灯慢慢变亮，再慢慢灭掉，循环往复，所以我们需要Compare值在0到99之间往复变化来改变PWM的占空比，进而控制给小灯的功率。至于为什么是0到99，首先它应该大于零，其次，这个compare值应当与我们刚刚设置的AutoReload值进行比较，而AutoReload的值为99，所以compare的值再大就没有意义了。

```
__HAL_TIM_SET_COMPARE(&htim3, TIM_CHANNEL_2, 0);


int compare = 0;
uint8_t flag = 1;
/* USER CODE END 2 */

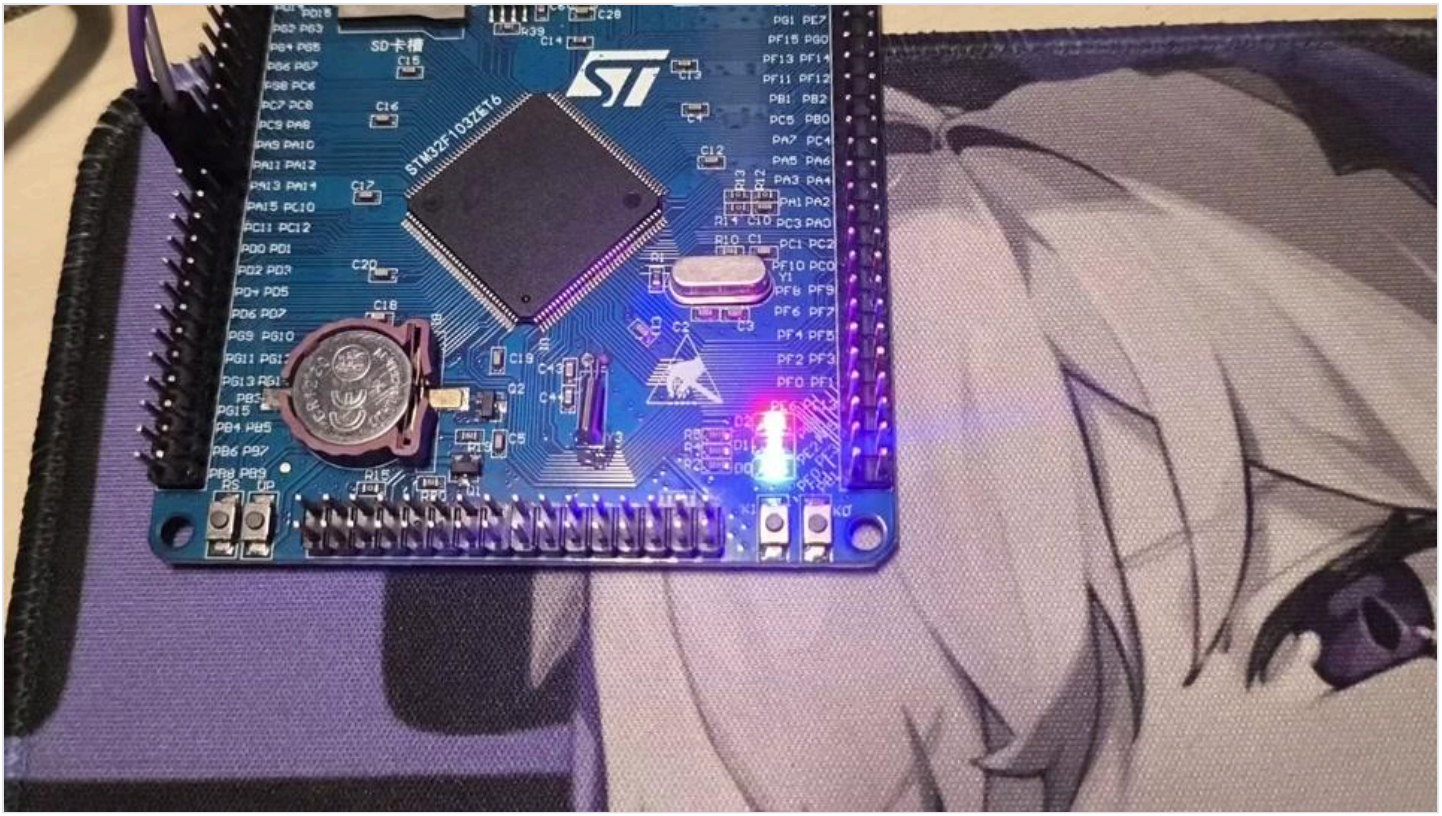
/* Infinite loop */
/* USER CODE BEGIN WHILE */
while (1)
{

  __HAL_TIM_SET_COMPARE(&htim3, TIM_CHANNEL_2, compare);


  if(compare>=100)
  {
    flag = 0;
  }
  else if(compare <= 0)
  {
    flag = 1;
  }
  if(flag)
  {
    compare++;
  }
  else
  {
    compare--;
  }
  HAL_Delay(Delay: 10);
  /* USER CODE END WHILE */

  /* USER CODE BEGIN 3 */
}
/* USER CODE END 3 */
}
```

编写完代码后，编译下载一气呵成，这样，我们就能做出属于自己的呼吸灯了



可以看到效果是led逐渐变亮又逐渐变暗

可以注意到的是，如果我们把compare值设定为一个固定的值，led就会发出一个固定的亮度，而这个亮度则由占空比来调节，占空比越大，灯越亮，占空比越小，灯越暗

以上是TIM的PWM最基本的使用方法，更多的内容就等各位去自行挖掘了

> 推荐拓展：去网上买一个无源蜂鸣器（用不了几块钱），用不同频率的pwm来驱动蜂鸣器实现让它发出不同的音调，用不同的占空比来控制它发出不同的响度，再稍加修饰，这样你就可以制作一个属于自己的音乐简谱播放器（bushi）了。不过这里仅做拓展，因为驱动电机，获取速度，和驱动舵机的任务有些繁重，大家可能没时间做这个了。（都看到这里了就给我这篇培训点个赞吧）

1、PWM

- 通过配置定时器模式，实现串口发送字母a增加LED的亮度，发送字母d降低LED的亮度，发送字母z实现LED的呼吸灯效果

提示：配置区别不大，主要在于改变PWM的占空比，结合上述文本或者资料中对占空比的叙述，查找HAL库改变占空比的函数；并且结合前面教程中对串口的使用。

2、编码器模式

- 尝试定时器的编码器模式，配置cubemx，然后写一下获取电机角速度和电机控制的代码。

提示：

代码块

```
// 电机转过一圈产生的脉冲数计算：
// 电机的基础脉冲 * 电机的减速比 *4
// 解析：电机基础脉冲为电机一转产生的脉冲数
// 而电机的减速比则会让电机内部转若干圈，电机外部才会转过一圈，
// 所以实际上电机外部转过一圈产生的脉冲数为  电机的基础脉冲 * 电机的减速比
// 而代码用的是 __HAL_TIM_GET_COUNTER，
// 在 STM32 的编码器模式下，定时器默认会对 A、B 两相的上升沿和下降沿都计数（即 4倍频）。
// 查阅产品手册可得，减速比为90， 电机基础脉冲为 11
constexpr float ENCODER_COUNT = 11 * 90 * 4; //电机转过一圈产生的脉冲数


constexpr float DT = 0.001f;//控制周期dt为1ms
constexpr float PI = 3.1415926535f;

// 编码器结构体定义
typedef struct
{
  TIM_HandleTypeDef tim;
  int32_t count;
  float radps;
  bool direction;
}encoder;

// 速度获取函数
void get_speed(encoder* encoder)
{
  encoder->count = __HAL_TIM_GET_COUNTER(&encoder->tim);

  if (encoder->count >= 32768)
  {
    //说明是向反向旋转
    encoder->count -= 65536;
  }
  encoder->direction = __HAL_TIM_IS_TIM_COUNTING_DOWN(&encoder->tim);

  //速度计算
  encoder->radps = (float)encoder->count / ENCODER_COUNT * 2 * PI / DT;  // 电机的角速度，单位：rad/s

  __HAL_TIM_SET_COUNTER(&encoder->tim, 0);

}
```

3、舵机驱动

- 同理，通过配置定时器模式实现舵机的角度控制，这个比较容易实现

网上自行查找舵机的原理和控制方法，这类教程还是很多的。但更建议查相应的数据手册，学会查找数据手册或者用户手册解决问题。

视频教程：【【STM32入门教程-2025】舵机，超清晰动画讲解】
https://www.bilibili.com/video/BV1mvpee4ECx?vd_source=559e497a1e15d78c1c0d7052a7ada06d

舵机原理

六、思考题

1. 已知 STM32 通用定时器 TIM3 挂载于 APB1 总线，系统配置 APB1 预分频系数为 2（即 PCLK1=36MHz），因此 TIM3 的时钟 TIMxCLK=72MHz。若需让 TIM3 每 500ms 触发一次中断，需配置预分频器 PSC 和自动重装载寄存器 ARR 的值（假设不考虑其他因素）：
   - 请根据文档中的定时时间公式（Timeout = ((PSC+1)×(ARR+1))/TIMxCLK），计算 PSC 和 ARR 的一组合理值（要求 PSC 和 ARR 均为 16 位以内整数）；
   - 解释公式中 “PSC+1” 和 “ARR+1” 的原因，并说明若省略 “+1” 会导致定时时间产生何种偏差。
2. 请设计基于 STM32CubeMX 和 HAL 库的 “LED 1 秒频闪” 功能（定时器中断实现），梳理完整配置流程，需包含以下关键步骤：
   - 时钟树配置：如何选择时钟源（如 HSE）并将系统主频配置为 72MHz，TIMx 时钟如何推导；
   - 定时器参数配置：选择哪个定时器（如 TIM2），PSC、ARR 的值如何设定（需匹配 1 秒定时）；
3. 文档中提到 L298N 电机驱动模块可通过 “单PWM+双IO口” 控制电机，若需实现以下功能，请分别列出 IN1、IN2和PWM口的电平 / PWM 状态：
   - 电机 A 正转且调速；
   - 电机 A 反转且调速；
   - 电机 A 制动；
   - 文档推荐 PWM 频率为 500~1000Hz，说明为何频率过低（如 50Hz）会导致电机控制效果变差。

——By 卢相泽

qq： 2071353315